Computers & Security 157 (2025) 104543

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Model-based structural and behavioral cybersecurity risk assessment in system designs

Tino Jungebloud [a,*], Nhung H. Nguyen [b], Dan Dongseong Kim [b], Armin Zimmermann [a]

[a] SSE Group, Technische Universität Ilmenau, Ilmenau, 98693, Thüringen, Germany
[b] School of Electrical Engineering and Computer Science, The University of Queensland, Brisbane, 4072, Queensland, Australia

ARTICLE INFO

Keywords:
Cybersecurity
Model-based systems engineering
Attack graph
UML
Graphical security modeling

ABSTRACT

Cybersecurity risk assessment has become a critical task in systems development and the operation of complex networked systems. However, current state-of-the-art approaches for detecting vulnerabilities, such as automated security testing or penetration testing, often result in late detection. Thus, there is a growing need for security by design, which involves conducting security-related analyses as early as possible in the system development life cycle.

This paper proposes an integrated approach that combines static and dynamic hierarchical model-based security risk assessment. The approach enables early identification of security risks during system design, utilizing various models based on the Unified Modeling Language (UML), with lightweight extensions using profiles and stereotypes to capture security attributes like vulnerabilities and asset values. These security attributes are then used to compute relevant properties, including threat space, possible attack paths, and selected network-based security metrics. To facilitate dynamic security analysis, the UML model is subsequently translated into a deterministic and stochastic Petri net (DSPN). This translation allows for the dynamic analysis and simulation of the system's state and behavior during an attack, capturing temporal aspects and probabilistic transitions. By representing system components and their interactions as modular Petri nets, the DSPN framework facilitates comprehensive simulation and analysis of possible attack scenarios. This also allows us to estimate time-based security metrics such as the duration required for an attacker to compromise system components. Consequently, this combined approach effectively addresses both static security analysis and dynamic state behavior, providing an integrated understanding of the system's resilience against cyber threats. A real-world industrial case study illustrates the effectiveness of this approach. The underlying data originates from security assessments performed by Keen Security Labs, which were independently verified by BMW (Cai et al., 2019). Specifically, we present an infotainment system network model as implemented in multiple car models along with corresponding attack and defense models. We then demonstrate how the approach assesses the cybersecurity risk of such in-vehicle networks.

1. Introduction

Cybersecurity has become an increasingly important topic in the development and operation of all kinds of computing systems. Protecting confidential data, guaranteeing data integrity and availability and maintaining system functionality are essential goals of cyber security. One common approach to analyzing cybersecurity-related properties of systems is information security risk assessment. All activities and efforts targeting risk assessment aim to answer the following questions: (i) *what attack scenarios can happen in a given context*; (ii) *what impact can a particular attack trigger, and how severe are attacks*; (iii) *how to respond to the identified risks,* and finally (iv) *how risks should be treated with respect to monitoring*.

The current state of the art in systems and software security includes automated security testing, such as penetration testing (often referred to as a 'pentest') and dynamic application security testing (DAST). However, these tests are often conducted only after development activities are completed or when the system is already in a productive environment. Therefore, this approach can lead to severe undetected vulnerabilities and the opportunity for remote exploitability. This is especially problematic in critical systems that human health or life may depend on, such as vehicles. Addressing these issues later can be challenging and expensive and may damage an organization's reputation and finances. Existing works demonstrated the potential impact of these issues. For example, Nie et al. (2017) showed in their study how

* Corresponding author.
E-mail address: tino.jungebloud@tu-ilmenau.de (T. Jungebloud).

https://doi.org/10.1016/j.cose.2025.104543
Received 10 November 2024; Received in revised form 30 April 2025; Accepted 23 May 2025
Available online 11 June 2025

an attack chain could compromise a Tesla Model S network by injecting malicious CAN messages. Another example is given by Cai et al. (2019) by demonstrating how to exploit vulnerabilities in BMW cars to gain full control of them. The vulnerabilities that led to unauthorized control of the car in these examples were due to many reasons, but one of them was the lack of security concerns in the system design process. However, assessing security risks during the system design and development process is a challenging task. When the system is not yet fully implemented, it is hard to identify all potential vulnerabilities and attack paths.

To address this problem, we propose a novel approach to start basic security analyses in the system development process as early as possible. Our method integrates both static and dynamic security modeling. The dynamic aspect leverages Petri nets to model system behavior and security properties over time. This allows for a detailed assessment of how the system responds to various attack scenarios and security events. Our static approach utilizes a UML-based model-driven engineering process, where system models are annotated with lightweight extensions using stereotypes. The dynamic model is derived automatically from the static specification and predefined Petri net models. We chose UML-based modeling for its widespread use, and its flexibility to be extended with security annotations. Similarly, Petri nets were selected for their suitability in modeling concurrency, timing, and stochastic behaviors critical to cyber–physical systems. This combination bridges the gap between practitioners' need for lightweight, integrable security assessment tools and the academic demand for formal, quantitative cybersecurity analysis. Overall, this approach supports proactive cybersecurity assessment without requiring engineers to be Petri net experts, making it accessible for a broad range of system development professionals.

Our framework allows System Architects & Designers, Software & Systems Engineers, and Cybersecurity Analysts to evaluate cybersecurity risks early in the system design process. This helps proactively identify and address potential vulnerabilities. Early assessment also enables security engineers to evaluate different defensive strategies systematically. This capability is especially valuable in complex, interconnected systems, where a single change can significantly affect the entire architecture.

An earlier version of this paper appeared in Jungebloud et al. (2024). This part of the methodology relies on UML models only and considers components, their interconnections and static network metrics providing early-stage security threats and risks identification, it falls short in modeling dynamic behaviors, such as attack propagation over time. To address these limitations and enhance the depth and applicability of the analysis , we have extended the previous work by (1) Integrating deterministic and stochastic Petri Nets (DSPN Ajmone Marsan and Chiola, 1987) into the early system design process. Petri nets offer a formalism that goes beyond the static UML models by incorporating dynamic aspects such as timing and sequencing of attacks and how system components are connected to each other under attack; (2) Introducing dynamic security and performance metrics for DSPN systems, specifically Attack Start Rate, Asset Compromised, Mean Time To Compromise (MTTC), providing quantifiable insights into system resilience and enhancing the design decision process; (3) expanding the case study evaluation by incorporating a dynamic model and conducting a comparative analysis of different attacker profiles to demonstrate how attacker capability influences the system's compromise time and dynamic risk behavior; (4) Providing a more detailed explanation of how each phase of the case study is modeled, along with a discussion of the results.

To the best of the authors' knowledge, this is the first proposal integrating hierarchical attack models with UML and DSPNs for comprehensive security analysis. The novelty of this paper lies in the integrated combination of static, model-based security analysis (using lightweight UML) with dynamic analysis through deterministic and stochastic Petri nets (DSPNs), enabling both early-stage and behavior-based risk assessment. Additionally, we demonstrate the practical applicability of the approach using a real-world automotive case study based on independently verified data.

The main contributions of this paper are summarized as follows:

1. Introduction of a seamless hierarchical model for assessing security risk, enabling early security evaluation in system development.
2. Support for the OMG UML standard for maximum interoperability with existing system models and other UML-based development toolchains.
3. Development of an automated algorithm to transform the UML model into graphs, using quantitative metrics to construct a threat space and score builder. This allows for the automated calculation of the overall security posture of the static system.
4. Dynamic security behavior modeling using Petri nets to evaluate the system's response to attack scenarios and attacker profiles. This includes introducing the MTTC metric for hierarchical analysis of security risks over time.
5. Evaluation of the effectiveness of the proposed model in a real-world in-vehicle network, as demonstrated in a case study conducted by Keen Security Lab at Tencent in 2019 (Cai et al., 2019).

The rest of this paper is organized as follows: Section 2 presents a review of related literature. The proposed approach is described in Section 3. In Section 4, we apply our approach to an automotive in-vehicle network as a case study. Section 5 outlines the limitations of our work and suggests future directions. Finally, we conclude this paper in Section 6.

2. Related work

In this section, we briefly discuss the existing related literature in three areas: (1) Cybersecurity risk assessment using OMG UML/SysML, (2) security modeling and analysis using stochastic Petri nets, and (3) Other Security Models and Metrics

2.1. Security models using UML/SysML

SysML Version 1 (OMG, 2019) is an extension of UML (OMG, 2017), which is used for specifying and analyzing complex systems. Apvrille and Roudier (Roudier and Apvrille, 2015) introduced the SysML extension called *SysML-Sec* with concepts and constructs to support secure software and hardware components. SysML-Sec adds security-specific constructs to UML specifications, including threat models, security goals, security requirements, security constraints, and security mechanisms. Pedroza (2019), Pedroza and Mockly (2020) employed SysML to conduct security risk analyses. The method relies on the principles of safety and security co-engineering and is aligned with the EUROCAE method specifications, including ED-203A (EUROCAE, 2014) and ED-202A (EUROCAE, 2018). Specifically, SysML block diagrams are used for architecture modeling, and profile extensions enrich models with security information. This can then be used to compute an attack path from an attack vector to a safety–critical component. Furthermore, the approach supports the identification of attack scenarios with MITRE CAPEC (MITRE, 2023a) for attack patterns and MITRE CWE (MITRE, 2023b) for software weaknesses.

2.2. Security modeling and analysis with Petri nets

Security modeling using Petri nets has become increasingly important due to its capability for quantitative analysis of cybersecurity risk. They allow the modeling of complex attack scenarios and defensive strategies. Petri net models with (stochastic) timing information

provide insights into metrics such as time-to-compromise (TTC) and mean-time-to-compromise (MTTC), which are essential for assessing a system's resilience against cyber threats.

McQueen et al. (2006) introduced a foundational TTC model to quantify cybersecurity risk. Using a Petri net framework, they modeled the time required for an attacker to compromise system components. This model considers various attacker's skill levels and vulnerabilities. An *overall time-to-compromise* is calculated as a weighted mean of the expected times for each attack sub-process, with weights assigned based on the likelihood of each process. This approach laid an early groundwork for quantifying risk based on attacker capabilities and system vulnerability.

Leversage and Byres (2008) extended the work of McQueen et al. by introducing an MTTC metric. Their model uses a state-space approach that divides attack progress into breach, penetrate, and strike states and includes algorithms for estimating attack paths and state times. By applying these algorithms, the model calculates the MTTC as a comparative security metric, allowing an evaluation across different configurations and mitigation actions. This approach provides a more detailed assessment of defensive measures' effectiveness over specific time intervals.

Mendonça et al. (2022) applied Petri nets to model Moving Target defense (MTD) strategies in Software Defined Networks (SDNs). Their model captures key phases of a cyber attack and evaluates metrics such as Attack Success Probability and MTTC under MTD conditions. This work highlights how stochastic Petri nets can assess dynamic defenses by altering network configurations.

Dalton et al. (2006) proposed the use of Generalized Stochastic PNs (GSPNs) for modeling Attack Trees (ATs) to enhance attack analysis with automated simulations via tools like PIPE (Bonet et al., 2007). This method uses GSPNs to translate Attack Trees into a probabilistic model to calculate attack probabilities. It offers a structured framework for visualizing and refining attack strategies and defensive measures.

Lathrop et al. (2003) propose MAADNET (Military Academy Attack/Defense Network), which is a methodology for simulating network attacks and defenses primarily for education purposes. MAADNET employs a modified attack tree model, including stages such as reconnaissance, exploitation, and consolidation. Each stage is assigned conditions and probabilities to reflect real-work attacker decision-making. While this model effectively captures dynamic attacker actions and varying skill levels, it falls short in providing detailed component-level transitions and key metrics like MTTC.

2.3. Other security models and metrics

Shaked and Reich (2021) proposed a model-based approach for designing cybersecurity-related systems, which considers both cybersecurity threats and system composition. The methodology introduces a domain ontology and a high-level modeling concept to aggregate components into hierarchies, which is similar to the modeling approach used in SysML. The system structure and the allocation of threats to single components are represented using a box-like notation. The methodology helps incorporate cybersecurity into systems design by identifying desirable security controls and associating them as functional capabilities at different levels based on threats and risks.

Another popular method for modeling systems and system security is using an attack graph (AGs) and an attack tree (ATs). Hong and Kim (2012) proposed two layers of Hierarchical Attack Representation Models (HARM) to evaluate the cybersecurity of enterprise networks. Their work combined both AGs and ATs to construct a comprehensive model to calculate the attack risk and attack cost of an update phase of a network. There are numerous works extending HARM to meet the characteristics of different networks (Enoch et al., 2021a). For example, HARM can be used to assess the effectiveness of the system when applying several different Moving target defense (MTD) techniques (Hong and Kim, 2016) to model the security of different types of networks, such as IoT networks (Ge et al., 2017, 2021) or maritime vessel networks (Enoch et al., 2021b).

An extension of ATs by dynamic fault trees are Attack-Fault Trees (AFTs), as introduced by Groner et al. (2023). Their method integrates manually created safety models (fault trees) with automatically generated security vulnerability information (attack trees), which are mined from vulnerability databases. They use deployment and dataflow models to bridge the abstraction gap between these separate models. Unlike regular fault trees, their approach supports stochastic behaviors, allowing descriptions of fault and attack rates, which facilitate probabilistic analysis. Similarly, Pekaric et al. (2023) also presented a fragment-based approach for the streamlined generation of attack graphs from publicly available security databases, employing a Domain-Specific Language to structure vulnerability information systematically. Compared to these works, the main distinction of our approach is the explicit utilization of existing UML-based architectural models as a starting point. Our approach provides a standardized, intuitive, and integrated entry point at the early stages of system design. Our approach leverages these existing UML models to automatically generate dynamic security analysis using DSPNs rather than combining separate manually-created safety and security models. Furthermore, our hierarchical approach enables capturing complex dynamic behaviors, including state-dependent operations and long-term steady-state analysis, which is a noted limitation in fault-tree-based and purely fragment-based methods.

Another approach to modeling system security is to use qualitative methods. Monteuuis et al. (2018) presented the Security Automotive Risk Analysis Method (SARA) for systematically analyzing threats and assessing risks of autonomous vehicles. This framework takes into account various factors that can impact the network security of the system, such as human factors, infrastructure sign recognition, the ability to control the car, and the severity of attacks. The security risk of the system was defined by a risk matrix function that considers factors including attack likelihood, controllability of the car, and attack severity.

2.4. Summary

The literature review presents various approaches for security modeling, attack path identification, attack modeling, attack scenario simulation, and risk assessment. However, some methods do not consider the system design or provide a means of modeling the system during the development process. Furthermore, some methods require modifications to the standard UML notation or necessitate starting over to conduct security analysis.

While existing studies on security modeling using Petri nets provide valuable insights, they also have several limitations. These models often lack dynamic, component-level progression within attack paths. They also omit detailed probabilistic transitions that capture diverse attack outcomes such as success, retry, or abort, as well as time-based parameters for each transition, both of which are critical for realistic modeling.

To address these gaps, we propose an approach that uses standard UML with profile extensions for system modeling and risk assessment, allowing an analysis based directly on the model. For dynamic security analysis, we introduce a Dynamic Attack Path (DAP) model based on DSPNs, incorporating detailed component states, probabilistic transitions, and timing parameters. This framework enables the calculation of MTTC metrics via Little's Law and provides a more adaptable representation of complex attack scenarios and defensive responses.

In the following section, we present a comprehensive overview of our proposed approach, outlining its key aspects and methodology.

Table 1
Symbols, definitions, and functions.

Symbols	Definition
Graph & Tree	
$\mathbb{N}$	Set of all components/nodes in the system, $\mathbb{N} = \{n_1, n_2, n_3, \dots, n_i\}$
$\mathbb{V}$	Set of all vulnerabilities in the system
v_j	Vulnerability j of the system, such that $v_j \in V$
α_{v_j}	Impact of vulnerability v_j
$Paths$	Set of *Attack Paths* in the system. Paths between an *attack entry point* and an *attack target*.
$Surface$	Attack Surface (set of *attack entry points*), $\Delta = \{\delta_1, \delta_2, \dots, \delta_n\}$, where $\Delta \subseteq \mathbb{N}, \delta_i \in \mathbb{N}$
$Assets$	Set of all *Assets* in the system (attack target), $\eta_i \in \mathbb{N}$
$T - Space$	Threat Space: Set of possible *attack entry point* to *attack target* combinations → $Surface \times Assets = \{(x, y) \mid (x \in Surface) \wedge (y \in Assets)\}$ (Cartesian product)
UML Model Views (Diagrams)	
$\mathbb{D}_{csd}^{H_i}$	Composite Structure Diagram at model hierarchy level i. Shows the internal structure of a classifier, classifier interactions with the environment through ports, or behavior of a collaboration.
$\mathbb{D}_{class}$	Class Diagram. Shows the static structure of classifiers and their relationships in a system, structural features (attributes), behavioral features (operations)
UML Model	
$\mathbb{M}$	Set of all model elements of type `uml::Element`.
$\mathbb{C}$	Set of all Classifiers in model $\mathbb{M}$ used as Components in one of the Composite Structure Diagram $\mathbb{C} = \{c_1, c_2, \dots, c_i\}$
$\mathbb{P}$	Set of Encapsulated Classifier attributes in the UML model used as ports of Components and parts in the Composite Structure Diagram $\mathbb{P} = \{p_1, p_2, \dots, p_j\}$
$\mathbb{E}$	Set of all UML Connectors in model $\mathbb{M}$. Connections between Component ports $\mathbb{E} \subseteq \{\mathbb{P} \times \mathbb{P}\}$
c_i	One component in the system or sub-system, $c_i \in \mathbb{C}$
p_j	One port classifier attribute of a component c_i. $p_j \in \mathbb{P}$
DSPN Model	
DAP	Dynamic Attack Path Model. $DAP = (C, R)$ where C is the set of components on an attack path and $R \subseteq CxC$ is the set of directed relationships between components, indicating potential paths an attacker can follow.
c_j	$c_j \in C$. A single component with states, transitions, arcs, probabilities, and timing parameters. $c_j = (P_j, T_j, I_j, O_j, P_j, Delay_j)$, where P_j Places, T_j Transitions, I_j Input arcs, O_j Output arcs and $Delay_j$ Timing parameters.

3. Proposed approach

This section outlines the workflow of our hierarchical model-based security risk assessment along with the dynamic models and security metrics used in the process. We introduce the general modeling approach using UML (OMG, 2017), the program interfaces and tools used to build the model and conduct the security assessment in a set of phases of the workflow subsequently. The section also presents the intuition and semantics of used paradigms, applied algorithms and transformations, and the selected metrics for conducting security assessment at each phase. To ensure a clear and unambiguous technical description, we introduce the definitions, symbols and functions in Table 1.

3.1. Hierarchical modeling workflow

Our workflow consists of four phases, as shown in Fig. 1: (1) Graphical system modeling using a UML modeling tool, parameterization, and transformation into executable C++ source code; (2) Model analysis for collecting key properties for further workflow steps; (3) Calculation of the *Threat Space* and the determination of *Attack Paths* using static intermediate tree/graph data structures and graph algorithms; and (4) Calculation of selected metrics (e.g., on the Attack Path level as well as on the Network level) to quantify cyber-security risk via a transformation of the static model into a stochastic Petri net. Next, we describe each phase in detail.

3.1.1. Phase 1: Graphical system modeling, configuration, and model transformation

In this phase, we use UML Designer (OBEO, 2023; SSE, 2023b), a free and open-source UML modeling tool based on Eclipse Sirius to create graphical models of the relevant system architecture. Structural parts of the system are described with their security-relevant parameters as well as connections between them that may allow attackers to proceed through the system.

Our approach relies on the UML meta-model and composite structure diagrams as graphical representations. To ensure that we can generate source code, apply security analysis, and facilitate the framework MDE4CPP (Maschotta et al., 2022; Hammer et al., 2022) later on, we define a precise way to model security components as follows: (1) Composites with `uml::Part` attributes are used for functional decomposition. (2) The concept of *Port on Part* utilizes strongly typed `uml::Connector` to model logical and physical communication channels between entities of type `uml::Encapsulated Classifier`. (3) The communication channels are modeled using Class Diagrams, including *Interface Classes*, *Required Interface*, and *Provided Interface*.

Our approach uses a hierarchical decomposition (c.f. Fig. 2) as follows: (1) At the highest hierarchy level (H0), system modeling begins by defining the *Security Scope*. This is done by adding an entity of type `uml:Class` to represent the *System Environment* along with other entities of the *System Environment*. For example, H0 may include two UML entities: `uml::Class` representing the *Attacker*, and `uml::Class` representing the *System* itself. (2) Every sub-system is then modeled as the next hierarchy level ($H1, H2, H3, \dots, Hn$), which includes the entities that require analysis later on. For example, a higher-level functional block represented as a `uml::Class` can be added to the model repository, followed by adding `Class::ownedAttribute` to the relevant component. In this way, during the system development process, more hierarchy levels can be added to provide a full model of the system if needed. The way to break down the functions into higher levels depends on the available information at the time of modeling and the analysis to be performed later. Fig. 2 illustrates the connection between different hierarchy levels as well as the diagrams used for the case study in Section 4.

Once the system modeling is complete, a profile extension is applied to the model. New stereotypes are defined along with the Metaclass relationship, as shown in Fig. 3. These stereotypes enable adding semantics to the model with numeric and text values and named enumeration constants. Each stereotype property has a descriptive name for improved understandability. Specifically, components in the system are annotated with stereotypes `CSElement`, `CSAsset`, and `CSMeasure`, while ports can have stereotype `CSPort`. All stereotype values added to the model are later used in algorithms and/or security metrics.

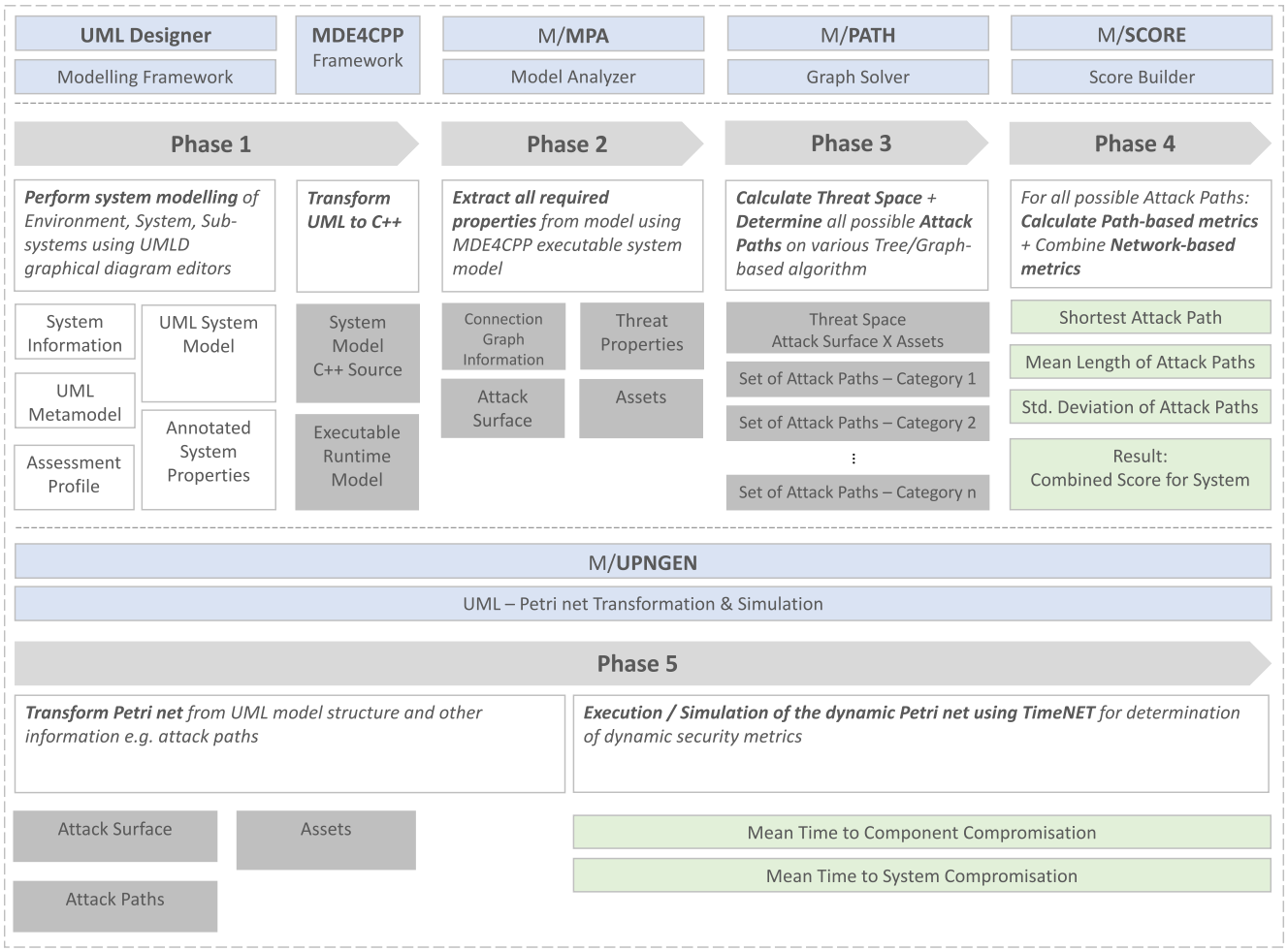

Fig. 1. A Workflow for a hierarchical model-based cyber-security risk assessment. Phase 1: Modeling with UML and transformation, Phase 2: Model analysis, Phase 3: Attack path identification, Phase 4: Network-based metrics, Phase 5: DSPN transformation and attack path simulation.

Next, the UML model is transformed into executable C++ source code using the MDE4CPP (Model-Driven Engineering for C++) (SSE, 2023a) framework. MDE4CPP is a model-driven engineering framework designed explicitly to facilitate the automatic generation of fully executable applications from UML models. We have extended MDE4CPP by incorporating specific UML-based security components such as hierarchical security functions, security-specific ports, security metrics, and communication channels. Additionally, we introduced custom annotations through stereotypes (*CSElement*, *CSAsset*, *CSMeasure*, and *CSPort*). These annotations, in combination with the internal MDE4CPP:uml4cpp generator, enable accurate translation of annotated UML models into executable C++ code. One critical challenge in using MDE4CPP is ensuring that the input UML model is accurate, complete, and correctly annotated. Errors or ambiguities within the UML specification (such as missing stereotypes, undefined attributes, or incorrect associations) can lead directly to compilation errors or incorrect execution behaviors in the generated code. Therefore, manual refinement and validation are typically necessary steps.

3.1.2. Phase 2: Model analysis and intermediate data structures

As a second step, the UML model from the previous phase is used to create a graph data structure for later analysis. To achieve this, we use the MDE4CPP:uml4cpp generator, which outputs C++ interface headers and implementation sources for every model element linked into shared libraries. This includes the complete UML meta-model, enabling versatile model analysis on both meta-model and model layers.

input : UML meta-model based system model $\mathbb{M}$.
result : Data structures representing the network topology of the system as a graph $G = (V, E)$; $Surface \subseteq V$; $Assets \subseteq V$.

Initialize: $V, E, Surface, Assets = \emptyset$;
parts ← `FindAllUMLParts()`;
foreach $v_i \in$ parts **do**
  $V \leftarrow v_i$;
  $p_i \leftarrow$ `FindPortsOnPart(`v_i`)`;
  foreach $v_j \in$ parts$\backslash v_i$ **do**
    $p_j \leftarrow$ `FindPortsOnPart(`v_j`)`
    if `CompareInterface(`$p_i[k]$,$p_j[k]$`)` **then**
      $E \leftarrow e(v_i, v_j)$;
  end
end

Algorithm 1: Algorithm for Phase 2: UML Model Analyzer (M/MPA).

The model analysis algorithm referred to as *M/MPA*, is developed in C++ to enable access to the object model in Phase 1. The details and principles of this algorithm are outlined in Algorithm 1. The required input is the UML model itself, denoted by $\mathbb{M}$. The algorithm initializes internal variables for handling graph elements `Node` and `Edge`. It then creates an instance of the system by calling the MDE4CPP model factory. Method `::FindAllUMLParts()` is executed at this instance level, resulting in applying a set of `uml::Class` elements

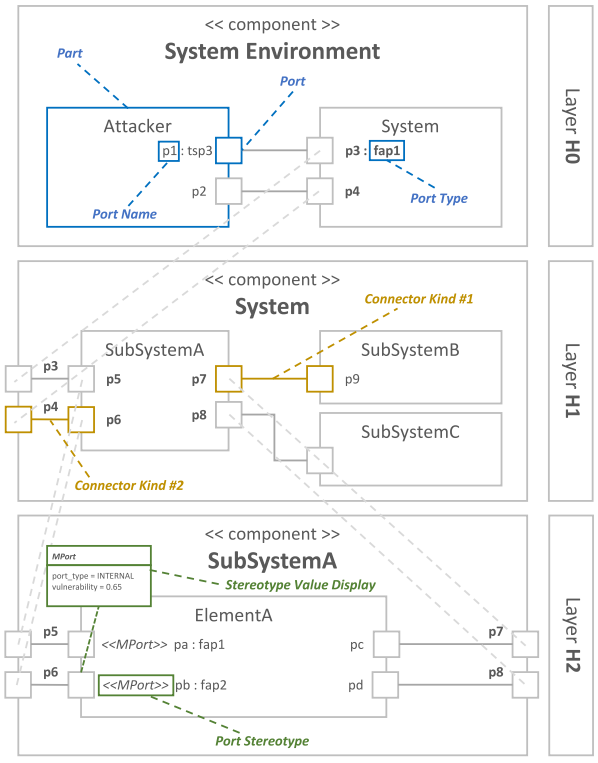

Fig. 2. Hierarchical decomposition with UML composite structure diagrams and stereotype visualization.

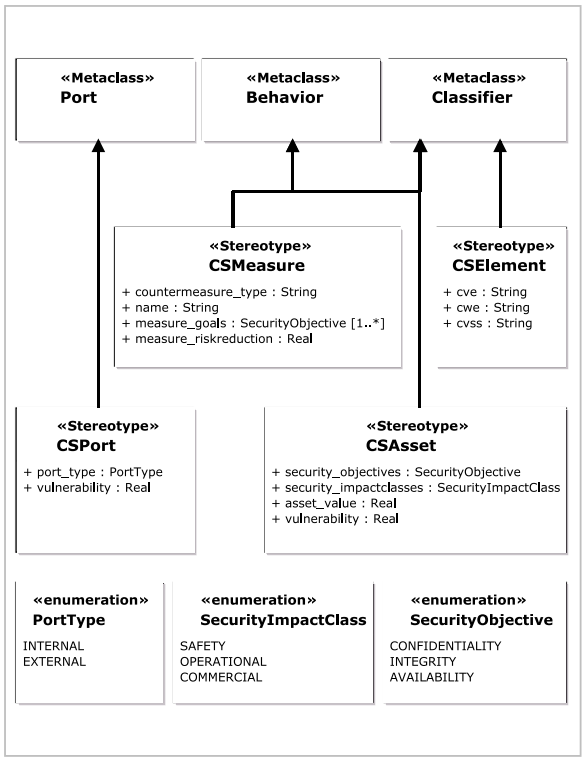

Fig. 3. Extract of the Risk Assessment UML profile including meta-class relationships.

Table 2
Comparison of BFS, DFS, and additionally the Dijkstra and A-Star search algorithms for systematic attack path determination.

	Search algorithm			
	BFS	DFS	Dijkstra	A-Star
Finds Shortest Path Efficiently	•	○	•	•
Memory Usage	High	Low	High	High
Implementation Effort	Low	Low	High	High
Time Complexity	$\mathcal{O}(V+E)$	$\mathcal{O}(V+E)$	$\mathcal{O}(E \log V)$	$\mathcal{O}(E \log E)$
Applicable to:				
Hierarchy	•	○	•	•
Weighted Paths	○	○	•	•

with a `Structured Classifier::part:Property` role. In the next step, the interface matching loop is run on all *Part* pairs in the system. If method `::CompareInterface()` detects a communication pair, the corresponding edge $E \leftarrow e(v_i, v_j)$ is added. Finally, the graph data structure G, as well as the lists of $Surface$ and $Assets$ elements, are written to separate files in the commonly used data interchange format JavaScript Object Notation (JSON).

3.1.3. Phase 3: Calculation of threat space and attack paths

In this phase, two tasks are conducted as outlined in Algorithm 2. The $ThreatSpace = Surface \times Assets$ is calculated first, which is the Cartesian product of the two lists `Surface`, `Assets` obtained from the result of the previous phase. The *Threat Space* can be interpreted as a list of possible end-to-end relations that start at components representing a possible attacker entry point and end at components that require protection against the loss of one or more security goals (→ *confidentiality*, → *integrity*, → *availability*). Essentially, it helps to identify potential security threats and vulnerabilities in the system by representing all possible combinations of the $Surface$ and $Assets$ elements.

The algorithm then proceeds to determine *Attack Paths* by reading the graph data structure from phase 2 and executing graph-based algorithms, namely Breadth-First Search (BFS) and Depth-First Search (DFS) traversing the graph to find all path(s) from an entry point to a target node. Finally, all information is collected and prepared to calculate metrics in the later steps. When analyzing attack paths in a network, BFS and DFS are fundamental algorithms used to explore possible routes an attacker might take. Each algorithm has distinct advantages depending on the network structure and the type of attack being analyzed. BFS explores a graph level by level, meaning that it examines neighbor nodes before moving down into the hierarchy. It produces useful results in terms of the number of hops from an entry point (`Surface`) to a target component (`Assets`). It is also well-suited for flat networks with a small number of connections per hierarchy level ensuring an optimal and efficient search. DFS, on the other hand, explores one potential attack path as deeply as possible down the hierarchy levels before backtracking. It is especially effective in uncovering deeply concealed attack paths, where an attacker uses multiple hops to reach an `Asset`. From a performance perspective, it requires less memory than BFS, making it more efficient for networks with a large number of connections between components. A comparison of BFS, DFS, and additionally the currently not applied Dijkstra and A-Star search algorithms is given in Table 2.

3.1.4. Phase 4: Calculation of security metrics and overall network security risk

This is the final phase of the hierarchical model. Here, security metrics are used to calculate risk scores on both the *Attack Path* levels and *Network* level. We obtain a selection of path-based metrics (cf. Enoch et al., 2020, p. 12) to analyze the attack paths. These path-based metrics provide valuable insight into network attack risk. The *Number of Attack Paths* (NAP) reflects the overall attack surface, with more paths indicating greater exposure. The *Shortest Attack Path* (SAP) highlights the quickest route an attacker can take through the network, where a lower value suggests a more immediate threat. *Mean*

Table 3
Network-based metrics and dynamic performance metrics.

Metric name	Abbrev.	Formula
Number of Attack Paths	*NAP*	$\mid Paths \mid$
Shortest Attack Path	*SAP*	$\min_{\forall Path \in Paths} \mid Path \mid$
Mean of Attack Path Lengths	*MAPL*	$\frac{\sum_{Path \in Paths} \mid Path \mid}{NAP}$
Standard Deviation of Attack Path Lengths	*SDPL*	$\sqrt{\frac{\sum_{Path \in Paths} (\mid Path \mid - MPM^2)}{NAP}}$
Mean Time to Compromise	*MTTC*	$\frac{L}{\lambda} = \frac{1 - \#\text{Compromised\_Node}_n}{\#\text{SystemRecover}}$

Attack Path Length (MAPL) gives an overall sense of how easily an attacker can navigate the network, with shorter averages implying higher risk. Meanwhile, the *Standard Deviation of Attack Path Lengths* (SDPL) captures variability—low values suggest consistent difficulty across paths, while high values may indicate weak points that are easier to exploit. Together, these metrics help assess how resilient a network is to potential attacks. The used path-based metrics, including their mathematical formulas, are summarized in Table 3.

3.2. Dynamic model for attack paths using Petri nets

The hierarchical model provides a foundational structure to represent the system, enabling automatic transformation into an executable model. It facilitates the calculation of threat spaces and the identification of potential attack paths within a static framework. However, this static approach assumes that all values and the model itself remain fixed and unchanging over time. In this section, we extend the hierarchical model to incorporate a dynamic perspective by introducing attacker profiles and system modeling using DSPNs.

3.2.1. Attack path modeling with Petri nets

A deterministic and stochastic Petri net (DSPN Ajmone Marsan and Chiola, 1987) is an extension of basic Petri nets that incorporate both deterministic and stochastic timing elements. A deterministic and stochastic Petri net is formally defined as a tuple DSPN, where $DSPN = (P, T, \Pi, Pre, Post, Inh, Delay, W, m_0, RV)$. P is the set of places, represented by circles, denoting states or conditions within the system. T specifies the set of transitions as the active parts of the model, which come in three varieties: deterministic transitions that fire after a fixed time, exponential transitions that fire after an exponentially distributed random time, and immediate transitions. Π is the priority of a transition as a natural number, which is zero for timed and a positive value for immediate transitions. *Arcs* serve as directed links between places and transitions, showing the flow of tokens: *Pre* denotes the multiplicities of input arcs (from places to transitions) and *Post* denotes the multiplicities of output arcs (and vice versa). *Inh* denotes the multiplicities of inhibitor arcs that disable a transition if there are at least as many tokens in place as the arc cardinality. The *Delay* value of a transition specifies the (average) firing delay that has to be passed after enabling the transition to be fired. It can either be zero for immediate transitions, a fixed value for deterministic transitions, or the inverse of the firing rate Λ for transitions with an exponentially distributed firing delay. W is the firing weight of an immediate transition (with firing time zero) as a real number. *Tokens* can be present in places representing the current state (marking) of the system. m_0 denotes the initial marking of the model. RV specifies the set of reward variables of a model (Zimmermann, 2008), i.e., the set of measures we are interested in calculating.

The semantics of a Petri net follow two rules: (1) a transition is *enabled* when there are enough tokens in input places and it is not disabled by any inhibitor arc (and there is no enabled transition with a higher priority). (2) An enabled transition can *fire* when it has been enabled as long as specified by its delay. A state change occurs by removing tokens from input places and adding tokens to output places.

With their capacity to model concurrent and time-dependent behavior, DSPNs are particularly well-suited for modular dynamic systems. In our proposed approach, a dynamic DSPN model is automatically generated from the attack path information collected in Phase 3, to execute a time-based security analysis.

3.2.2. Model definition

A **Dynamic attack path model (DAP)** is defined as $DAP = (C, R)$, where C are the components in the attack path and $R \subseteq CxC$ is the set of directed relationships between components, indicating potential paths an attacker can follow from one component to another.

Each component $c_i \in C$ has its own set of states, transitions, arcs, probabilities, and timing parameters $c_i = (P_i, T_i, I_i, O_i, W_i, Delay_i)$, where:

P_i **is a set of possible Places**. For a given component c_i, Places is defined as $P_i = \{NoAttack, UnderAttack, AttackTrialEnd, Compromised\}$. Here, $NoAttack$ is an initial state where the component is not under any attack; $UnderAttack$ is the state where the component is actively being attacked. $PostAttack$ is the state where the component has undergone an attack, but there is still the probabilistic choice modeling if the attack was successful or not; $Compromised$ is the state where the component has been fully compromised by the attack.

T_i **is a set of Transitions** between Places. For a given model component c_i, the transitions to be generated are given by $T_i = \{\tau_{\text{start}}, \tau_{\text{trial}}, \tau_{\text{success}}, \tau_{\text{fail}}, \tau_{\text{abort}}, \tau_{\text{recover}}\}$. Arcs between transitions and places are created for the following input–output place flows and encoded in I_i and O_i:

$\tau_{\text{start}} : NoAttack \rightarrow UnderAttack$
$\tau_{\text{trial}} : UnderAttack \rightarrow AttackTrialEnd$
$\tau_{\text{success}} : AttackTrialEnd \rightarrow Compromised$
$\tau_{\text{fail}} : AttackTrialEnd \rightarrow UnderAttack$
$\tau_{\text{abort}} : AttackTrialEnd \rightarrow NoAttack$
$\tau_{\text{recover}} : Compromised \rightarrow NoAttack$

The set of probabilities W_i associated with immediate transitions is set as follows: $W_i = \{w_{\text{success}}, w_{\text{fail}}, w_{\text{abort}}\}$, where each probability (or weight) w corresponds to specific transitions. Specifically, w_{success} aligns with τ_{success}, setting the probability that a trial will succeed, which leads to the compromised state. w_{fail} aligns with τ_{fail}, presenting the probability that a trial will fail, which returns to the under-attack state. w_{abort} corresponds to τ_{abort}, presenting the probability of aborting the attack after a failed attempt, which returns to the no-attack state.

$Delay_i$ **is a set of timing parameters** for the transitions of one subsystem: $Delay_i = \{delay_{\text{MTTAttackStart}}, delay_{\text{MTTSingleTrial}}, delay_{\text{MTTReset}}\}$, where each timing parameter is associated with specific transitions. Specifically, $delay_{\text{MTTAttackStart}}$ aligns with τ_{start}, representing the meantime to attack start when the component is idle. $delay_{\text{MTTSingleTrial}}$ aligns with τ_{trial}, is the mean time between single trials, representing the persistence of the attack process. $delay_{\text{MTTReset}}$ aligns with τ_{reset}, representing the meantime to reset the system back to the initial state after full compromise.

input : Network topology as graph $G = (V, E)$; *Surface* $\subseteq V$; *Assets* $\subseteq V$.
result : *T-Space*: $\{(x, y) \mid (x \in \textit{Surface}) \wedge (y \in \textit{Assets})\}$; *Paths*: $\{P_i\}_{i=1..n} \mid P_i = (v_1, ..., v_N) : (v_i, v_{i+1}) \in E$ for $1 \leq i \leq N - 1$.

Initialize: tspace, paths $\leftarrow \emptyset$;
Initialize: graph $\leftarrow G$;

Step 1: Calculation of Threat Space;
tspace $\leftarrow$ surface $\times$ assets;

Step 2: Calculation of Paths;
forall *pair* $\in$ tspace **do**
  start $\leftarrow$ *pair.first*;
  target $\leftarrow$ *pair.second*;
  paths $\leftarrow$ paths + `doPathSearch(graph, start, target)`;
end

Algorithm 2: Algorithm of Phase 3: Graph Solver (M/PATH).

3.2.3. Model description

Fig. 4 shows the generic modular DSPN model proposed in this paper for one attack path, following the model definition explained before. The attack path DAP has n components $C = (Node_1, Node_2, \ldots, Node_n)$.

The system starts with one token in `NoAttack_Node_1`, modeling the idle state. An attack occurs with rate $1/delay_{\text{MTTAttackStart}}$ (i.e., on average after a time of $delay_{\text{MTTAttackStart}}$). The token in the place `NoAttack_Node_1` presents the system state where the system has not been attacked. Once an attacker starts trying to breach the system, the transition τ_{start} fires, moving the token from `NoAttack_Node_1` to `UnderAttack_Node_1`, indicating that the first node is now under attack. The system then proceeds with a sequence of attack trials, where each trial occurs at the rate of $1/delay_{\text{MTTSingleTrial}}$, corresponding to transition `AttackTrial_Node_1`. After each trial, the transition `AttackTrial_Node_1` fires and moves the token to the place (modeling the state) `AttackTrialEnd_Node_1`.

Based on the outcome of each trial, the system either transitions to the state `PostAttack_Node_1`, signifying a successful attack at Node 1, or remains in `UnderAttack_Node_1` for another trial. If the attack is successful, the token moves to `Compromised_Node_1` with a probability w_{success}. If the attack fails, the token is moved to `UnderAttack_Node_1` with a probability w_{fail}, and another trial begins. If the attacker gives up or aborts the attack, the state is transferred to `NoAttack_Node_1` with probability w_{abort}.

Once the attack at $Node_1$ is completed and the node compromised, the transition τ_{start} activates for $Node_2$, and the same process repeats for $Node_2$. This pattern continues until the attack has propagated through all n nodes of the attack path, with each node undergoing similar attack transitions.

Such a model would always end up in the final state of the last node being compromised after an unknown time. Thus, it would only allow a transient analysis or simulation. To check long-term behavior and compute dynamic properties like the MTTC in steady-state, we thus assume that there is a random time after which the system operators detect the compromised state and do a recovery restoration. This is modeled by transition `SystemRecover`, which fires with a rate defined by $1/delay_{\text{MTTReset}}$. The `SystemRecover` transition returns all tokens to their initial, uncompromised state `NoAttack`.

3.3. Security metrics

3.3.1. Static security measures

Static security metrics (Enoch et al., 2020) as specified in Table 3 can be derived from the specification:

- The number of attack paths (*NAP*) counts the paths that an attacker can take to compromise the target in the system. The higher the number of attack paths, the less secure the system is from a qualitative perspective.
- The shortest attack path length (*SAP*) is the number of compromising steps on the shortest way for the attacker to reach the target.
- Mean of attack path length (*MAPL*) denotes the average length of all attack path. This metric shows the average effort that the attacker may have to apply to be able to reach the target.
- As the average length of paths *MAPL* may be misleading if the distribution of lengths covers a wide variety of values, also the standard deviation of the attack path lengths *SDPL* may be of interest to a system designer.

3.3.2. Dynamic security measures

While qualitative security measures based on a static architecture model may be helpful for a system designer, the actual efforts required to compromise certain parts of a system can vary greatly, and the effects of timed maintenance, etc., cannot be assessed in this way. We thus propose computing security measures from the stochastic timed Petri net model, which can be easily extended to incorporate more complex attacker and defender models and even evaluate the impact of concurrent attacks. The main metric we are interested in is the time required for an attacker to compromise an asset starting from an attack surface. As this is a random variable, we compute the *mean time to compromise* (MTTC) for a single attack path.

For a DSPN model created for an attack path following the previously described rules, the MTTC is calculated as the average time from the first node leaving the state `NoAttack` until the final node reaches the state `Compromised`. Such delays are not directly expressible with the standard performance measures, including rate and impulse rewards, which compute state probabilities (`#Place> 0`) or average marking numbers (`#Place`), and transition firing frequencies `#Transition`. However, we can apply Little's law (Little, 1961), which calculates these traversal times based on values that are easily computable.

Little's Law is a fundamental principle in queuing theory, defined by an equation $L = \lambda W$, where L presents an average number of items in a system and λ the average arrival rate of customers into the system, while W denotes the average time they spend in the system, the value that we are interested in. This relationship has been widely applied in telecommunications and computer science to measure system response times. In our cybersecurity model, Little's Law can be used to derive the MTTC: Instead of customers moving through a queueing system, we consider system states or tokens, such as `NoAttack`, `UnderAttack`, `AttackTrialEnd`, `Compromised` and the input transition into the system. By adapting Little's Law, we calculate the MTTC as:

$$\text{MTTC} = \frac{L}{\lambda} \tag{1}$$

where L presents the average occupancy of the compromised states in all nodes on the path and λ is the effective arrival rate into the considered subsystem, in our case, the throughput of transition `StartAttack_Node_2`. In a steady state, the throughput of this transition must match the system recovery rate, and the sum of the probabilities of internal nodes being in a compromised state, along with the final compromised state, must equal one. Thus, we can express the MTTC as:

$$\text{MTTC} = \frac{1 - \#\text{Compromised\_Node}_n}{\#\text{SystemRecover}} \tag{2}$$

This simplifies the formula as aborted attacks on the first node would be recounted wrongly, as we are only interested in the effective time until a successful attack finishes.

Here, $1 - \#Compromised\_Node_n$ presents the proportion of the attack path that is not compromised and $\#SystemRecover$ is the rate of system

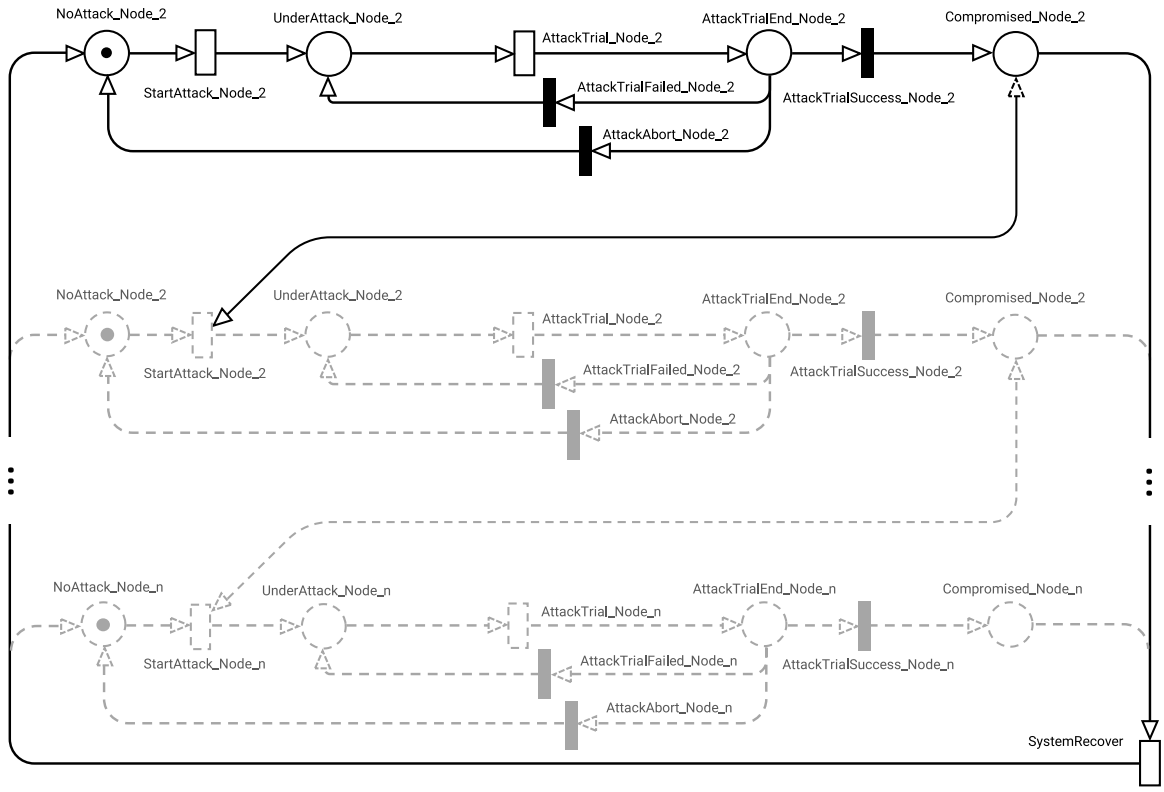

Fig. 4. Generic attack path model for the analysis of dynamic behavior during system attacks.

recovery from compromised states. It should be noted that this takes into account the mean times of attacks starting from the idle state, i.e., it is the mean time from the point of view of the system and not the average time that the attacker needs to fully succeed from the beginning of the attacks. This slightly different value can also be computed in a similar way by subtracting the probability of a token in the idle state from the $(1 - \cdots)$ formula. A higher MTTC value signifies greater system resilience, indicating that an attacker needs more time, on average, to fully compromise the system (including the assumed time between attacks).

4. Case study: BMW in-vehicle infotainment system

In this section, we describe the use of our proposed method and metrics using an example of an automotive in-vehicle network. This case study is based exclusively on publicly available information published by Keen Security Lab in 2019, which BMW independently verified (Cai et al., 2019). The choice of this case study was motivated by the richness of information available, including detailed system architecture, attack vectors, and known vulnerabilities. This level of detail is rare in publicly available studies, making it an ideal candidate to demonstrate the applicability and effectiveness of our method. While previous work by Keen Security Lab focused on discovering and verifying vulnerabilities, our work provides the first security modeling and risk analysis for this case study using a combined static and dynamic model-based approach.

We selected the automotive domain for our case study primarily because of its clear structure, comprehensive documentation, and practical significance to a broad audience, especially considering the increasing cybersecurity challenges in modern vehicles. Existing cybersecurity analyses for automotive systems were available, allowing us to focus directly on applying and validating our methodology without conducting extensive initial risk assessments from scratch. This automotive case study clearly demonstrates all key steps of our approach, providing a comprehensive illustration. Additionally, certain individual elements and preliminary concepts from our methodology have been explored in different domains, such as drone operations in the Real-Time Analytic, Prognostics, and Health Management (RTAPHM) (Zimmermann and Jungebloud, 2023) project, as well as smart manufacturing in the Engineering for Smart Manufacturing (E4SM) (Bedini et al., 2024) project. Due to the publication constraints of this project's system, we selected an automotive scenario (BMW) for this paper due to its clarity, completeness, increasing cybersecurity challenges in modern vehicles, and relevance to a broad audience.

Specifically, we (1) briefly present the case study with a infotainment system network model, attack model, and defense model and (2) implement a static security analysis using our proposed lightweight UML-based models. We then conduct a dynamic behavior analysis using deterministic and stochastic Petri nets. This implementation enables practitioners and researchers to systematically model, analyze, and simulate cybersecurity risks in complex networked systems early in the design process.

4.1. Vehicle network

An in-vehicle network is a complex system of embedded computer systems known as Electronics Control Units (ECUs), which are connected to each other using various internal networks and network protocols. Each ECUs can act as the hub of the network, connecting to various sensors and actuators throughout the vehicle. In addition, they can be used to manage specific functions of the car, including engine control, body control, infotainment, and safety functions such as the Anti-lock Braking System (ABS), as well as Advanced Driver Assistance Systems (ADAS), which utilize sensors like RADAR, LIDAR, and cameras.

ECUs are connected to different types of communication protocols based on their operation speed and functionality. These protocols include Controller Area Network (CAN), Local Interconnect Network

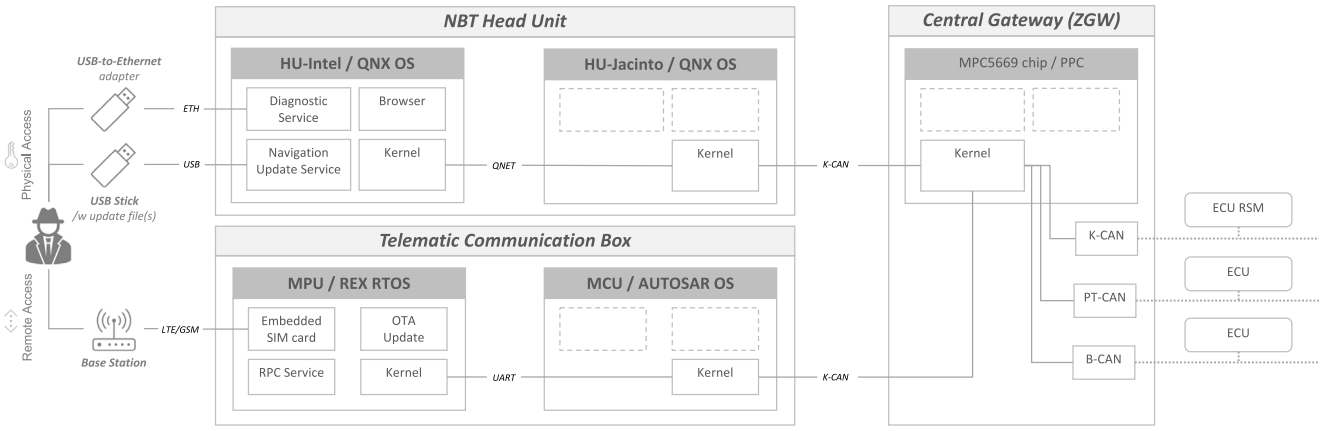

Fig. 5. An example automotive in-vehicle network with attack vectors/paths.

(LIN), Media Oriented Systems Transport (MOST), FlexRay, Ethernet, and others, but CAN is the dominant protocol in modern vehicles.

The vast majority of cars nowadays are equipped with an onboard diagnostic (OBD) protocol and an OBD-II port to diagnose the operation of the ECUs. By using an OBD dongle or OBD scanner that physically connects to the OBD-II port, the user or car manufacturer can communicate with a set of specific ECUs in the car and update their firmware.

Modern cars have connecting abilities that help them communicate with various networks and surrounding environments. These connections include common technologies such as WI-FI, Cellular (with built-in SIM card), Bluetooth, GPS, and USB. Most of these connections are embedded inside the informatics sections and are well known as the head unit and telematic unit of the car. These technologies provide vehicles with more capabilities and help broaden drivers' experience. On the manufacturer side, these connections offer new diagnostic options, such as firmware updates through the Internet (Over-The-Air update) and remote diagnostics using the Unified Diagnostic Services protocol (UDS). These technologies provide more features and capabilities for both drivers and manufacturers, but they also open a dozen new attack surfaces and make vehicles vulnerable to cyber-attacks.

4.2. System model

To demonstrate the practical applicability of the proposed model, we use the real in-vehicle architecture and attack scenarios that were previously introduced (Cai et al., 2019). Their work analyzed the architecture of a BMW car and provided a comprehensive security analysis with different attack vectors for the car. Fig. 5 shows an example in-vehicle network. Attackers can achieve their attack goals through multiple entry points including (1) USB-to-Ethernet adapter enabling Ethernet-based access; (2) USB Stick containing automatically loaded update file(s); and (3) LTE/GSM wireless access.

The main focus is on two ECUs of the car, the Head Unit (HU) and Telematic Unit (TU), which connect to the outside network of the car. These units serve as the main attack vectors that allow the attackers to potentially access the inside network of the car remotely or physically.

The HU has two main chips, which are Intel x86 and TiDra44x. The Intel x86 chip is responsible for the main operation of the head Unit, including displaying information, connecting to other networks, etc. For security reasons, the Intel x86 cannot directly send arbitrary CAN messages to the gateway. The CAN message can only be sent to the gateway through the TiDra44x.

Similarly, the TU also has two main chips. One is the Qualcomm MDM6260, which is responsible for all the remote services to the manufacturer's server, such as emergency calls, tele-diagnostic, remote services, and OTA update service. The other chip is FreeScales9512X, which is responsible for sending and receiving CAN messages and is directly connected to the Gateway.

The Gateway is running on an MPC5669 chip. It has multiple connections to different CAN BUS as well as FlexRay, LIN and MOST BUS. The Gateway chip has a special function of receiving diagnostic messages using UDS protocol from the HU and TU. The HU and TU are connected to Gateway using CAN-K BUS, which is dedicated to the entertainment domain only.

4.3. Attacker model

Our assumption is that the attacker's goal is to gain physical control over at least one of the car's ECUs, which could allow them to manipulate various aspects of the car's body (such as door, light, horn control) or more seriously, active or deactivate the car, or manipulate the airbag system. The attacker can use one of two approaches to achieve this goal: physical or remote attack.

Physical attack: An attacker can exploit vulnerabilities in the diagnostic or navigation services by using a USB (USB-to-Ethernet adapter; USB Stick) to gain root shell access to HU-Intel. Once this is compromised, the attacker can log in to HU-Jacinto and send CAN messages to ECUs on the K-CAN Bus. Furthermore, the attacker can send diagnostic messages to different CAN BUS from the QNX hu-intel x86 to control different ECUs.

Remote attack: The attacker can set up a rogue base station to force the car to connect to their network, using it as a gateway to access the HU and TU of the vehicle (CVE-2018-9311 (v_5)). The attacker can then compromise the Browser (CVE-2012-3748) (v_1)) and escalate privilege to get the root access to the Kernel of the Intel x86 chip (CVE-2018-9322) (v_2)). Using a rogue base station, the attacker can also remotely compromise the kernel of the Qualcomm Kernel of the Telematic Unit. With the root privilege to the Head Unit and Telematic Unit, the attacker is able to send diagnostic messages to different CAN BUS to control different ECUs. Specifically, in the case of a physical attack, the attacker can use the USB-to-Ethernet adapter to exploit the Time of Check to Time of Use (TOCTOU) weakness (CVE-2018-9322) (v_2)) in the diagnostic service, which can lead to the execution of bash commands with kernel root privilege. Alternatively, the attacker can use a USB stick with a malicious update file to exploit the Stack overflow weakness (CVE-2018-9312) (v_3)) in the navigation map update service, resulting in code execution with root privilege on the Intel x86's QNX OS. With weak authentication on TiDrax44's QNX OS (v_4)), the attacker can fully compromise the Head Unit and send arbitrary CAN messages as well as UDS diagnostic messages to control different ECUs in different CAN buses behind the Center Gateway. In the case of a remote attack, the attacker can exploit the Browser's outdated version of AppleWebKit/535.17, using the Memory corruption weakness (CVE-2012-3748) (v_1)) in the Webkit engine to gain shell access with browser

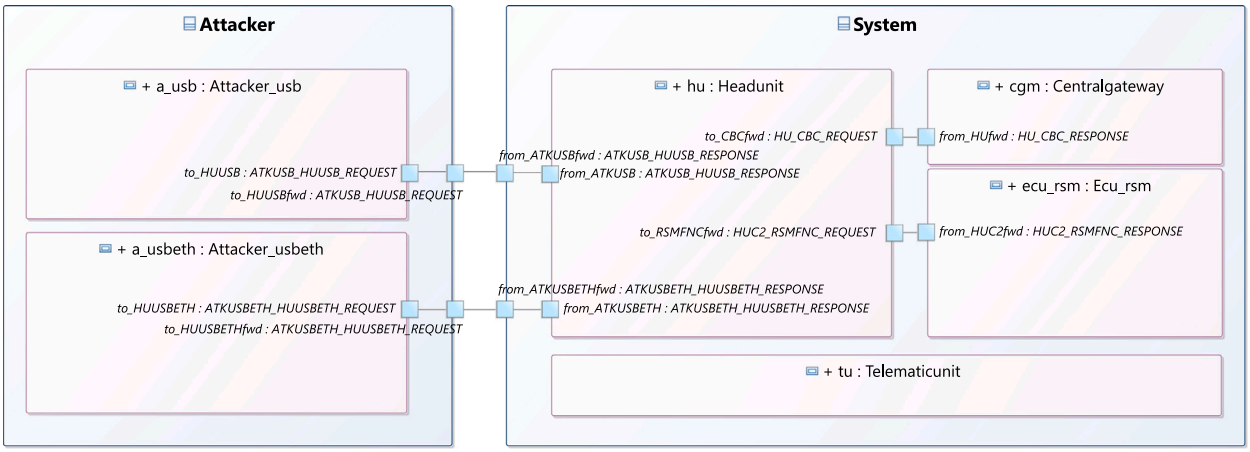

Fig. 6. System composite structure diagram $\mathbb{D}_{csd}^{H_0}$ (Hierarchy level 0, top-level/environment). of the example automotive in-vehicle network as depicted in Fig. 5.

Table 4
Vulnerabilities information in the in-vehicle network with Sequential Number, CVE ID, CVSS3 Base Score, Impact, Likelihood and CVSS3 Base Vector String.

Notations	CVE	CVSS	I.	LH.	CVSS3 Base Vector String
v_1	CVE-2012-3748	7.7	5.3	1.8	AV:N/AC:H/PR:L/UI:N/S:C/C:L/I:H/A:L
v_2	CVE-2018-9322	7.8	5.9	1.8	AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
v_3	CVE-2018-9314	6.8	5.9	0.9	AV:P/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
v_3	CVE-2018-9312	7.8	5.9	1.8	AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
v_4	–	6.7	4.7	1.5	AV:L/AC:L/PR:H/UI:N/S:C/C:N/I:H/A:L
v_5	CVE-2018-9311	9.8	5.9	3.9	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

privilege. Then, using the weakness in the diagnostic service described above (CVE-2018-9322) (v_2)), the attacker can gain root privilege of the Head Unit. Additionally, the attacker can use the vulnerability in remote service to send malicious Short Message Service (SMS) through the rogue base station and exploit the stack overflow in the Verify Signature function of the OTA update to gain root execution of Qualcomm kernel privilege. Similar to the Head Unit, the attacker can use this root privilege to send UDS diagnostic messages to control the ECUs on different CAN BUS.

Table 4 shows the details of the vulnerabilities described above. We use the Common Vulnerabilities and Exposures (CVE)/National Vulnerability Database (NVD) to collect information about the vulnerabilities. We use the metric of the base score from the Common Vulnerability Scoring System (CVSS) version 3.0 (CVSS Special Interest Group (SIG), 2025; Jungebloud, 2025) to acquire the value of impact and likelihood of the vulnerability. For vulnerabilities without a CVSS v3.0 score or CVE-ID, we manually analyzed and assigned appropriate scores based on the vulnerability characteristics. Specifically, we reviewed and assessed the vulnerability descriptions to estimate CVSS 3.0 metrics, including exploitability metrics (attack vector, attack complexity, privileges required, user interaction, and scope) and impact metrics (confidentiality, integrity, and availability). The detailed value behind these estimations is summarized clearly in the column *CVSS3 base vector string* of Table 4.

4.4. Defense model

The aim of the defense strategy is to achieve a level of security. To achieve this, we consider using two defense methods: (a) *Patch Management* to address vulnerabilities and weaknesses with software patches. However, closing some vulnerabilities may be complex and time-consuming, so we assume that only a few vulnerabilities can be patched. (b) *Intrusion Detection and Prevention (IDPS)* to monitor occurring events in the systems communication network and detect signs of possible incidents. However, the computational performance of in-vehicle components is limited. Therefore, only a reduced set of functions can be applied to specific types of attacks. We assume that an IDPS can only be applied at the Central Gateway to monitor CAN Buses to the connected ECUs or to prevent the HU and TU from sending malicious UDS messages.

4.5. Assess static security risk using hierarchical UML/SysML security model

4.5.1. System modeling, parameterization, and transformation

We use the system model and attack model information to conduct UML modeling. To emulate the entire system design process, we divide the modeling into three levels that show varying degrees of abstraction at different stages of the system design process. These levels are: environment layer $\mathbb{D}_{csd}^{H_0}$, system layer $\mathbb{D}_{csd}^{H_1}$ and sub-system layer $\mathbb{D}_{csd}^{H_2}$. Diagram $\mathbb{D}_{csd}^{H_0}$, the environment layer, describes the system in its considered surroundings. As shown in Fig. 6, this level includes `uml::Class` *Attacker* and `uml::Class` *System* with its external interfaces (→ *Threat Surface*) is modeled. Diagram $\mathbb{D}_{csd}^{H_1}$ are structural details of `uml::Class` *System* as shown in Fig. 7, which contain components `uml::Classs` *HeadUnit*, *CentralGateway*, *ECU_RSM*, and *TelematicUnit*. At this level, interfaces and *logical* connections between these components are also modeled. $\mathbb{D}_{csd}^{H_2}$ is a sub-system layer diagram and is created for *HU-Intel*, *HU-Jacinto*, *ECU_RSM* and others. The focus here is the functional decomposition. For *HU-Jacinto* the decomposition is composed of *DiagnosticService*, *NavigationUpdateService*, *Browser*, *KernelHUC1*, and `uml::Connectors` which are typed by `uml::Associations` to model communication interfaces. More hierarchy models $\mathbb{D}_{csd}^{H_i} | i \geq 3$ may exist depending on (a) the intended analysis detail level and/or (b) the complexity of the system.

4.5.2. Model analysis and intermediate data structures

In this step, Algorithm 1 is used for model analysis. The algorithm uses the hierarchical system model from the previous step as an input to generate the System Connection Graph and JSON files. The System

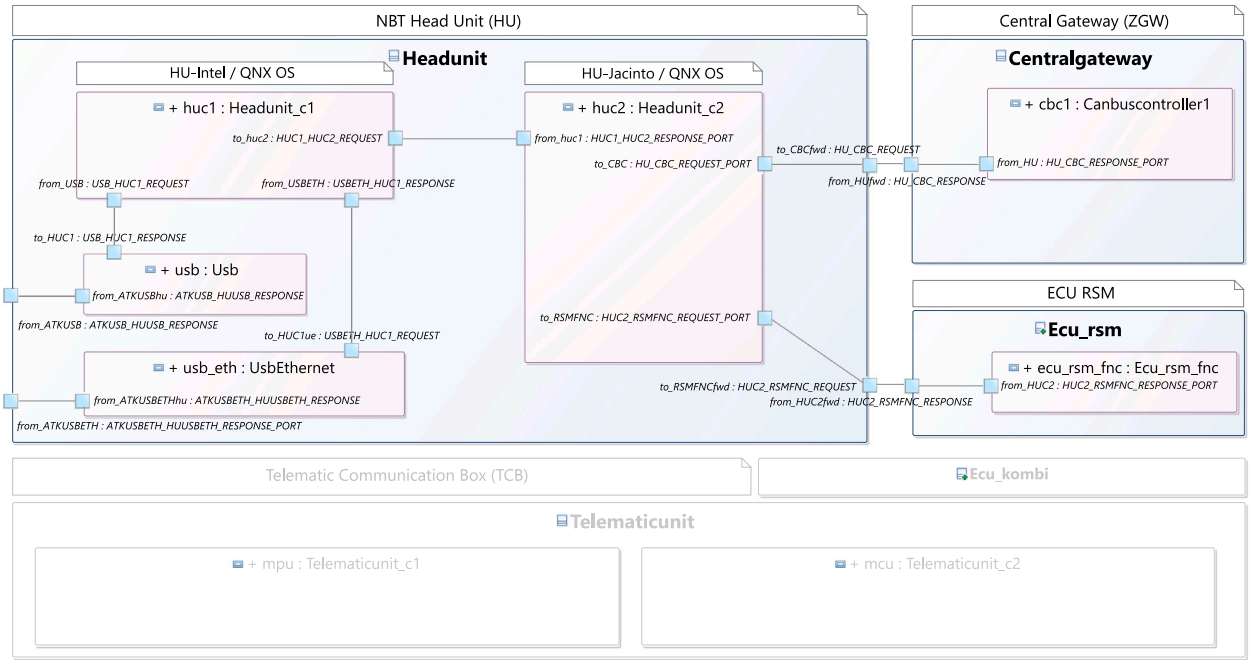

Fig. 7. System composite structure diagram $\mathbb{D}_{csd}^{H_1}$ (Hierarchy level 1, decomposition of composite *System*).

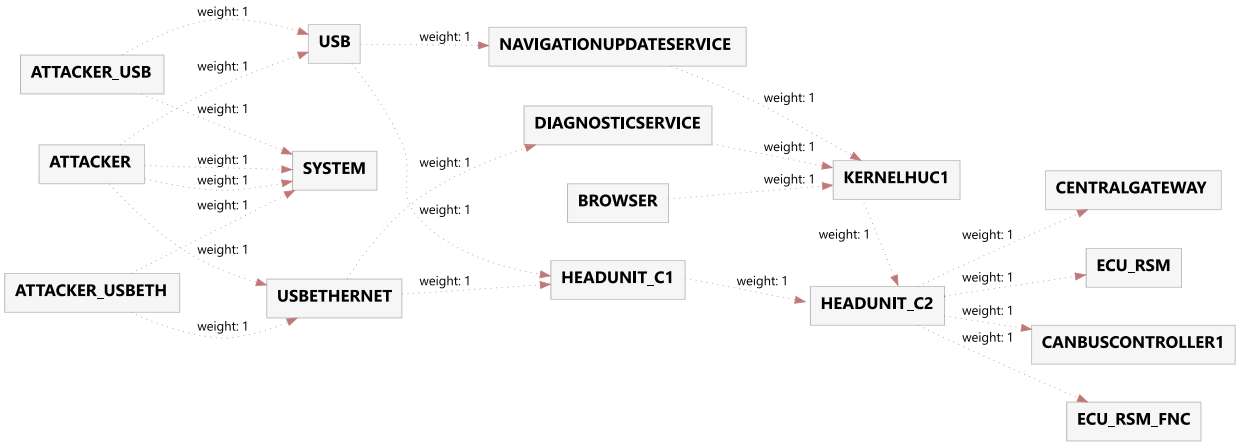

Fig. 8. System connection graph generated from the internal graph data structure. Directed graph, generated using Graphviz/dot. Connections between all elements considered in the static network attack path analysis.

Connection Graph visually represents the hierarchical system model, as shown in Fig. 8. This graph helps identify the connections between components and interfaces in the system. The JSON file contains the structure of the system in JSON format, represented as graph G. These files are then handed over to the next phase for further analysis.

4.5.3. Threat space and attack paths

In this case study, we use the following setting: The Attack Surface consisted of two attack scenarios *Attack Surface* ={ *ATTACKER_USB*, *ATTACKER_USBETH*} and the system had three assets *Assets* = { *ECU_RSM*, *ECU_RSM_FNC*, *KHUC1*} Algorithm 2 is then used to generate the corresponding *Threat Space* and identify all *Attack Paths*. The result data is stored as a JSON file and the set of *Attack Paths* are given in Table 5.

4.5.4. Security metrics and overall risk

In this step, the path-based metrics are calculated. We use the vulnerabilities information in Table 4 as an input for the metrics in Table 3. Specifically, the input values (vulnerabilities) for the network were as follows: *BROWSER* = 7.7 (CVE-2012-3748), *HU* = 7.8 (CVE-2018-9322), *HU_C1* = 7.8 (CVE-2018-9322), *KHUC1* = 7.8 (CVE-2018-9322), *HU_C2* = 6.8 (CVE-2018-9314), *TU* = 9.8 (CVE-2018-9311), *NUS* = 9.8 (CVE-2018-9312). Then, the security metrics are calculated as follows: *NAP* = 6; *SAP* = 4; *MPL* = 4.67; *SDPL* = 0.47; Finally, an overall risk score is calculated. Considering the six possible attack paths, the overall risk score is 13.63.

Let $x \in \mathbb{R}$ the risk score in the range 1–30, then we define the overall risk $f(x)$ as a piecewise definition as follows:

$$f(x) = \begin{cases} low & 1 \le x < 10 \\ medium & 10 \le x < 20 \\ high & 20 \le x \le 30 \end{cases} \tag{3}$$

According to Eq. (3) the overall risk is evaluated to *medium*.

4.6. Assess dynamic security risk using DSPN

This section demonstrates how dynamic security analysis can be performed using our DSPN model. We use the dynamic model to

Table 5
Analysis Results with AP Length (L), Asset Value (V), Score (S1), score when applying patching defense (S2), Score when applying IDS (S3), and AP components.

	L.	V.	S1	S2	S3	Attack path components
1	5	3.5	2.92	1.36	1.56	ATK_USB ⊳ USB ⊳ HU_C1 ⊳ HU_C2 ⊳ ECU_RSM
2	5	7.5	2.92	1.36	1.56	ATK_USB ⊳ USB ⊳ HU_C1 ⊳ HU_C2 ⊳ ECU_RSM_FNC
3	4	9.5	3.90	1.95	3.90	ATK_USB ⊳ USB ⊳ NUS ⊳ KHUC1
4	5	3.5	2.92	1.36	1.56	ATK_USBETH ⊳ USBETH ⊳ HU_C1 ⊳ HU_C2 ⊳ ECU_RSM
5	5	7.5	2.92	1.36	1.56	ATK_USBETH ⊳ USBETH ⊳ HU_C1 ⊳ HU_C2 ⊳ ECU_RSM_FNC
6	4	9.5	1.95	0.00	1.95	ATK_USBETH ⊳ USBETH ⊳ DS ⊳ KHUC1

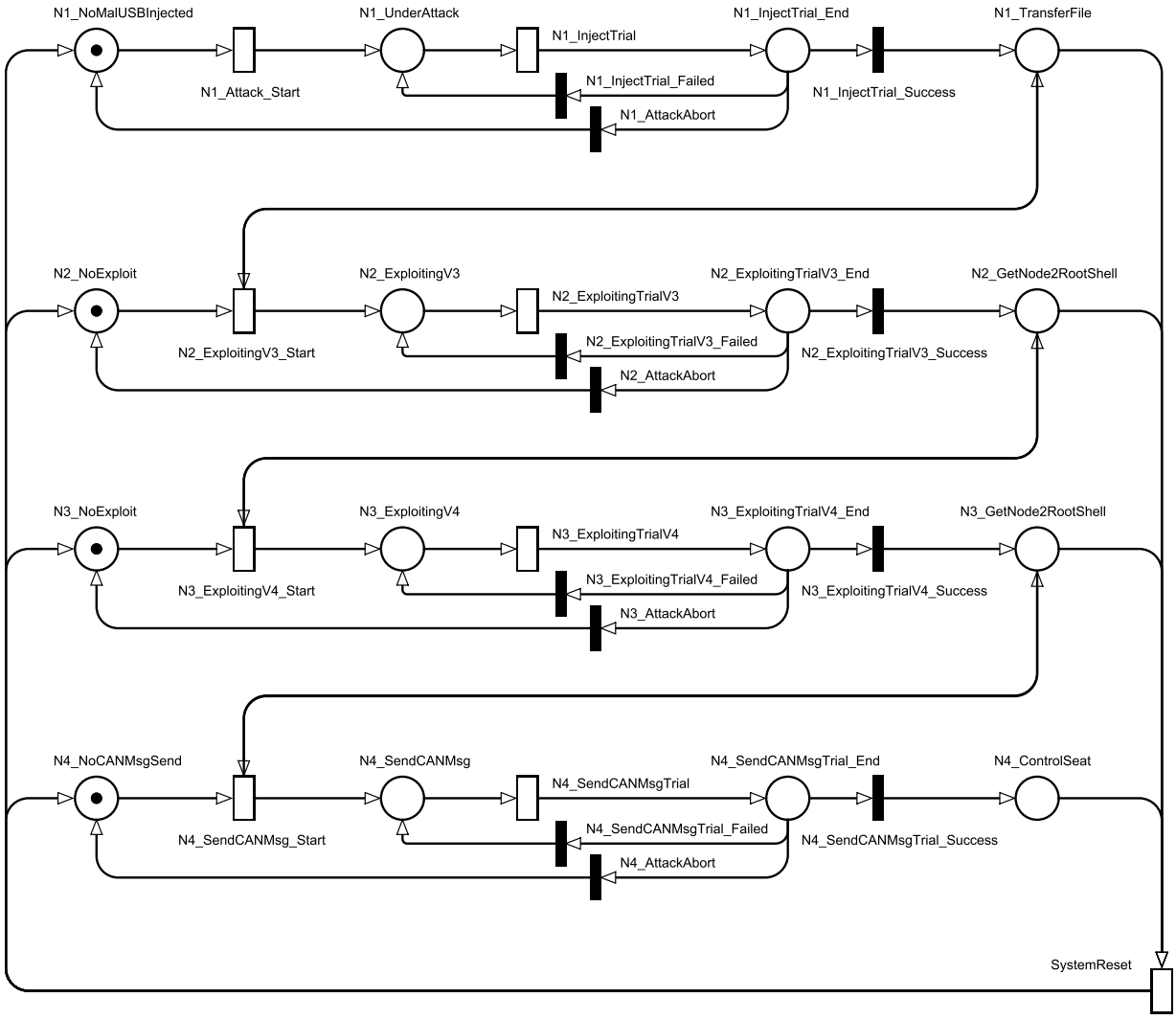

Fig. 9. DSPN model representing case study attack path No. 1, created using the Petri Net modeling framework TimeNET. Stochastic behavior and state transitions associated with the analyzed attack scenario.

illustrate not only the attack progression over time but also to conduct a comparative analysis of different attacker profiles. This allows us to simulate and quantify how attacker capabilities influence the system's resilience, particularly regarding compromise time and dynamic behavior under attack. The comparative analysis also validates that the model captures differences in attack success rates and timings depending on the attacker type, rather than reflecting only static vulnerabilities. To demonstrate our dynamic model using DSPN, we selected an identified *Attack Path* from the previous analysis. The chosen attack path is $ATK_USB \rightarrow USB \rightarrow HU_C1 \rightarrow HU_C2 \rightarrow ECU_RSM$. This path is modeled in the DSPN shown in Fig. 9.

4.6.1. Model description

In Fig. 9, the DAP is modeled as an attacker sequentially compromises nodes using multiple exploitation techniques. The model comprises four main nodes, each representing a distinct step in the attack sequence, followed by a `SystemReset` mechanism to restore the system to an uncompromised state after each complete attack cycle.

$Node_1$ represents the initial phase of the attack, where the attacker uses a USB injection technique to gain access to the system. The presence of a token in `N1_NoMalUSBInjected` enables the transition `N1_Attack_Start`, initiating the USB injection attempt. This transition, $\tau_{N1\_Attack\_Start}$, fires after the average time $delay_{\text{MTTAttackStart1}}$, representing the frequency of the attacker's USB injection attempts. Once $\tau_{N1\_Attack\_Start}$ fires, the model moves into `N1_UnderAttack`,

indicating an active injection attempt. The transition $\tau_{\text{N1\_InjectTrial}}$ simulates this attempt with probabilistic outcomes. If the injection succeeds (transition $\tau_{\text{N1\_InjectTrial\_Success}}$ fires with probability $w_{\text{n1\_success}}$), the token advances to `N1_TransferFile`, signifying that the attacker has successfully transferred a malicious file and can proceed to the next stage. If the injection fails (transition $\tau_{\text{N1\_InjectTrial\_Failed}}$ fires with probability $w_{\text{n1\_fail}}$), the token returns to `N1_UnderAttack` for another attempt. If the attacker decides to abort the attack, the token moves back to `N1_NoMalUSBInjected`, indicating that the system remains secure with no active attack.

If the attacker successfully injects USB and transfers malicious files to the system, the transition $\tau_{\text{N2\_ExploitingV3\_Start}}$ is enabled to fire after time $delay_{\text{MTTAttackStart2}}$, representing the frequency at which the attacker begins the exploitation process. Once $\tau_{\text{N2\_ExploitingV3\_Start}}$ fires, the token moves into `N2_ExploitingV3`, indicating an active attempt to exploit the system. The transition $\tau_{\text{N2\_ExploitingTrialV3}}$ simulates this exploitation trial with probabilistic outcomes. If the exploitation succeeds (transition $\tau_{\text{N2\_ExploitingTrialV3\_Success}}$ fires with probability $w_{\text{n2\_success}}$), the token advances to `N2_GetNode2RootShell`, signifying that the attacker has obtained root shell access of the HU-Intel and can proceed to the next stage. If the exploitation fails (transition $\tau_{\text{N2\_ExploitingTrialV3\_Failed}}$ fires with probability $w_{\text{n2\_fail}}$), the token returns to `N2_ExploitingV3` for another attempt. If the attacker decides to abort the attack, the token moves back to `N2_NoExploit`, indicating that the system remains secure and unaffected by the exploitation attempt.

If the attacker successfully gains root shell access in HU-Intel, the transition $\tau_{\text{N3\_ExploitingV4\_Start}}$ is enabled to fire after time $delay_{\text{MTTExploitingStart3}}$. This transition represents the start of an exploitation attempt targeting vulnerability v_4 in order to gain control of the Hu-Jacinto. Once $\tau_{\text{N3\_ExploitingV4\_Start}}$ fires, the token moves into `N3_ExploitingV4`, where the attacker actively attempts to exploit vulnerability v_4. The trial transition, $\tau_{\text{N3\_ExploitingTrialV4}}$, has probabilistic outcomes: if successful (transition $\tau_{\text{N3\_ExploitingTrialV4\_Success}}$ with probability $w_{\text{n3\_success}}$), the token advances to `N3_GetNode3RootShell`, indicating that the attacker has achieved root shell access on Head Unit component 2. If the attempt fails (transition $\tau_{\text{N3\_ExploitingTrialV4\_Failed}}$ with probability $w_{\text{n3\_fail}}$), the token returns to `N3_ExploitingV4` for a retry.

After successfully gaining root shell access on HU_Jacinto, the attacker moves to $Node_4$, which aims to control the vehicle's seat unit (RSM) by sending a malicious CAN message. The presence of a token in `N4_NoCANMsgSend` enables the transition `N4_SendCAN-Msg_Start`, which initiates the CAN message injection attempt. This transition, $\tau_{\text{N4\_SendCANMsg\_Start}}$, fires after time$delay_{\text{MTTAttackStart4}}$, representing the frequency at which the attacker attempts to send a control message to the seat unit. Once $\tau_{\text{N4\_SendCANMsg\_Start}}$ fires, the model moves into `N4_SendCANMsg`, indicating an active attempt to inject the CAN message. The injection trial transition, $\tau_{\text{N4\_SendCANMsgTrial}}$, has probabilistic outcomes: if successful, the transition $\tau_{\text{N4\_SendCANMsgTrial\_Success}}$ with probability $w_{\text{n4\_success}}$ fires, the token moves to `N4_ControlSeat`, signifying that the attacker has successfully taken control of the RSM unit. If the attempt fails (transition $\tau_{\text{N4\_SendCANMsgTrial\_Failed}}$ with probability $w_{\text{n4\_fail}}$), the token returns to `N4_SendCANMsg` for another attempt.

4.6.2. Attacker profile

To estimate the values for the set of probabilities and timing parameters within our DSPN model, we leverage attack profiles to represent varying levels of attacker skill and familiarity with the system. While multiple studies have proposed different attack profiles, we adopt the EVITA attack profiles (Olaf et al., 2009) due to their comprehensive classification of attacker expertise.

As shown in Table 6, EVITA profiles categorize attackers into three levels: Layman, Proficient, and Expert. Considering different attacker profiles allows the assessment to capture the full spectrum of potential adversary capabilities. A Layman may only exploit obvious vulnerabilities, while Proficient and Expert attackers might use advanced techniques to breach more secure systems. This tiered approach ensures that our attack simulations include both simple and sophisticated threats, providing more nuanced and effective evaluation results. Each profile provides a calculation factor that helps adjust probabilities and timing parameters according to the skill and knowledge level of the attacker. For example, an Expert attacker with extensive system knowledge and technical skills will have shorter trial times and higher success probabilities. In contrast, a Layman will have lower efficiency and success probabilities.

Table 6
Attacker profiles and corresponding calculation factors as proposed by EVITA. Olaf et al. (2009).

Profile	*Description*
Layman (0) [1]	Unknowledgeable compared to experts or proficient persons, with no particular expertise
Proficient (3) [3]	Knowledgeable in being familiar with the security behavior of the product or system type.
Expert (6) [6]	Familiar with the underlying algorithms, protocols, hardware, structures, security behavior, principles and concepts of security employed, techniques and tools for defining new attacks, cryptography, classical attacks for the product type, attack methods, etc.

4.6.3. Simulation setup and comparative analysis

With the generated SDPN model shown in Fig. 9 and these EVITA-defined profiles, we conduct simulations under three scenarios:

- Scenario 1: An *Expert* attacker with deep system knowledge, high proficiency, and low attack time intervals.
- Scenario 2: A *Proficient* attacker with moderate system knowledge and trial times between those of the *Layman* and *Expert*.
- Scenario 3: A *Layman* attacker with minimal system knowledge, requiring longer time intervals for each trial attempt.

The parameters used in the model are summarized in Table 7. For simplicity, two parameters remained constant across all simulations: *MTTReset* and *MTTAttackStart*. The *MTTReset* was set to 900-time units, representing the system recovery interval, and *MTTAttackStart* was set to 30-time units, indicating the mean time between consecutive, independent attacks.

Each simulation ran for a duration of 1000 time units, allowing for one complete recovery cycle within each run, as the recovery event occurs every 900-time unit. Other parameters were based on publicly available information and reasonable estimations intended to illustrate the application of our method. For attack paths and based vulnerability exploitation success probabilities, we referenced the public findings of Keen Security Lab, supplemented by CVSS-based interpretations where applicable. For attacker profiles, values such as Probability for each Attacker Profile were assigned based on logical estimations of attacker skill levels, as specified in the Attacker Profiles Table 6. The parameter *MTTSingleTrial*, representing the time between consecutive exploitation trials during an ongoing attack, varied by attacker skill level: an Expert attacker was assumed to perform a trial every 5-time units, a Proficient attacker every 10-time unit, and a Layman every 30-time unit.

Since real-world values vary depending on specific system conditions, undisclosed vulnerabilities, and differing analyst judgments, these values should not be interpreted as precise predictions for all system instances. In this case study, we selected parameters to demonstrate trends in attack timing, system compromise rates, and resilience behavior. Our methodology allows users to adjust parameters based on their specific system characteristics and threat models.

Simulations were conducted using the TimeNET 4.4 (Zimmermann, 2017) simulation framework. The transient analysis results are shown

Table 7
Simulation parameter for Attacker Profiles A_{exp} (Expert), A_{pro} (Proficient) and A_{lay} (Layman), used in TimeNET model simulation for Attack Path No. 1.

Parameter	Base Prob.	Probability			
		A_{exp}	A_{pro}	A_{lay}	mean
MTTReset	900	–	–	–	–
MTTAttackStart	30	–	–	–	–
MTTSingleTrial	–	5	10	30	15
t_get_into_car	.13	.78	.39	.13	.43
t_cannot_get_into_car	.87	.22	.61	.87	.57
t_stop_get_into_car	.15	.10	.55	.85	.50
t_exploit_v3_success	.08	.48	.24	.08	.27
t_exploit_v3_fail	.92	.52	.76	.92	.73
t_exploit_v3_abort	.15	.10	.55	.85	.50
t_exploit_v4_success	.10	.60	.30	.10	.33
t_exploit_v4_fail	.90	.40	.70	.90	.67
t_exploit_v4_abort	.11	.34	.67	.89	.63
t_send_can_msg_succ	.07	.42	.21	.07	.23
t_send_can_msg_fail	.93	.58	.79	.93	.77
t_send_can_msg_abort	.15	.10	.55	.85	.50

in Fig. 10, with x-y graphs presented as sub-figures. The left-hand graphs display the number of active attacks per component, while the right-hand graphs show the time series of component compromises. Next, we examine the simulation results for each component in the attack path, highlighting the differences in active attacks and time to compromise across the three attacker profiles: Expert, Proficient, and Layman.

USB Attack Surface/N1: The results for the first component in the attack path, the USB Attack Surface (Node 1), are shown in Figs. 10(a) and 10(b).

For *Expert* attackers, the dark line in Fig. 10(a) represents the average number of active attacks. Generally, experts move quickly from the initial attack surface to the next component of the attack path, as indicated by the lower curve in Fig. 10(a) and the rapid rise in Fig. 10(b). This reflects the Expert attacker's ability to compromise this component more quickly than other attacker profiles.

The *Proficient* attacker, represented by the turquoise line, takes more time to penetrate the attack surface due to moderate skill levels. This is shown by a higher curve in Fig. 10(a) compared to the Expert and a slower compromise progression in Fig. 10(b). Proficient attackers require roughly three times as long as Experts to fully compromise the component.

In contrast, *Layman* attackers remain at the attack surface node for the longest duration due to their limited skill. This is evident in the yellow line in Fig. 10(a), which has the highest values among the attacker profiles. As a result, Layman attackers progress more slowly toward compromise, with fewer successful attacks moving past the attack surface than Expert and Proficient attackers.

Head Unit QNX1/N2: Results for the second component, Head Unit QNX1 (Node 2), are shown in Figs. 10(c) and 10(d).

For *Expert* attackers, the active attacks displayed in Fig. 10(c) achieve success early, reflecting the rapid advancement of the attacker through this component. The probability of compromise reaches a peak around model time 200, indicating that the success rate $w_{\text{n2\_success}} = 0.48$ is reached after several exploit trials.

Proficient attackers, with moderate skills and parameter settings, take longer to compromise this component. As shown in Fig. 10(d), the Proficient attacker reaches the compromise peak later than the Expert, with maximum compromise occurring around 600-time units.

For *Layman* attackers, active attacks on this component appear significantly later than for other profiles. The peak level of active attacks, shown as 0.18 in Fig. 10(c), reached around 350-time units. Additionally, the component compromise curve in Fig. 10(d) is relatively flat due to a lower success rate of $w_{\text{n2\_success}} = 0.08$, reflecting the extended time required for the Layman attacker to achieve compromise.

Head Unit QNX2/N3: Results for the third component, Head Unit QNX2 (Node 3), are shown in Figs. 10(e) and 10(f).

For *Expert* attackers, active attacks occur early, around model time 100. *Proficient* and *Layman* attackers, with lower success rates of $w_{\text{n3\_success}} = 0.30$ and $w_{\text{n3\_success}} = 0.10$, reach higher compromise levels later, around model times 300 and 700, respectively. This delay reflects their slower progress due to difficulties in exploiting earlier components.

ECU RSM/N4: The results for attack path component 4, ECU RSM (Node 4), are shown in Figs. 10(g) and 10(h). In the final component, the behavior of all attacker profiles is similar to that observed in the previous component. For *Layman* attackers, the curve remains very flat, which can be attributed to their low success rate $w_{\text{n4\_success}} = 0.07$ and slow lateral progression. This slow rise contrasts sharply with *Expert* attackers, who achieve a steady level of active attacks early, peaking at about 0.08 around 100 system time units. Fig. 10(h) shows that *Expert* attackers quickly reach a high compromise level, stabilizing around a steady-state value of 0.81 by approximately 300 system time units. *Proficient* attackers achieve a lower steady-state compromise level of 0.59, reached after a slower progression that stabilizes around 600 system time units. In contrast, *Layman* attackers show a much slower and limited compromise progression, with a steady-state value of only 0.38 by the end of the simulation. Additional results are displayed in Fig. 10(i).

Thus, the simulation results follow a clear overall trend. In the initial phase, the Figures show how attack activities develop as system time progresses. If the mean of active attacks starts low across all expertise levels, it could suggest that initial system defenses effectively prevent attackers from executing numerous attacks. In the mid-phase, there are significant fluctuations in attack activity. If the *Expert* attackers exhibit a higher mean of active attacks compared to *Proficient* and *Layman* attackers, it indicates their ability to adapt more rapidly, exploiting system vulnerabilities that other attacker profiles struggle to access. Then, in the final phase, as the timeline approaches the end, the Figures illustrate whether the model stabilizes near a steady-state solution.

Finally, the MTTC measure, shown in Fig. 10(j), provides a comparative overview for each attacker profile. As expected, the *Expert* attackers have the lowest MTTC at 199, indicating their ability to compromise the system quickly. The *Proficient* attackers' MTTC is more than three times higher at 610, and the *Layman* attackers exhibit the highest MTTC at 1445, reflecting their limited capacity to compromise the system efficiently.

4.7. Application of defense models

Since the BMW vehicle system has been modeled and its security risks assessed, we now use the framework to test and analyze the effectiveness of various defensive strategies. This analysis demonstrates how the system's risk profile changes after implementing specific mitigation measures, helping us decide to select and deploy appropriate defenses.

To enhance the security of our system, we have implemented two defense strategies: patch management and the use of IPDS. We are measuring the impact of these strategies on the overall security score of the system through calculation and comparison. To analyze the impact of *Patching management*, we changed the input vulnerability value of components *HU*, *HU_C1* and *KHUC1* to 0.00. This can be interpreted as solving software issues related to CVE-2018-9322. The risk scores of affected Attack Paths are given in Table 5 column $S2$. The overall risk score of this defense model is 5.83. When using patch management, it would be ideal to patch all known and possible vulnerabilities. However, this is not practical in the real world, as patching can be costly and time-consuming. Therefore, we choose one vulnerability in

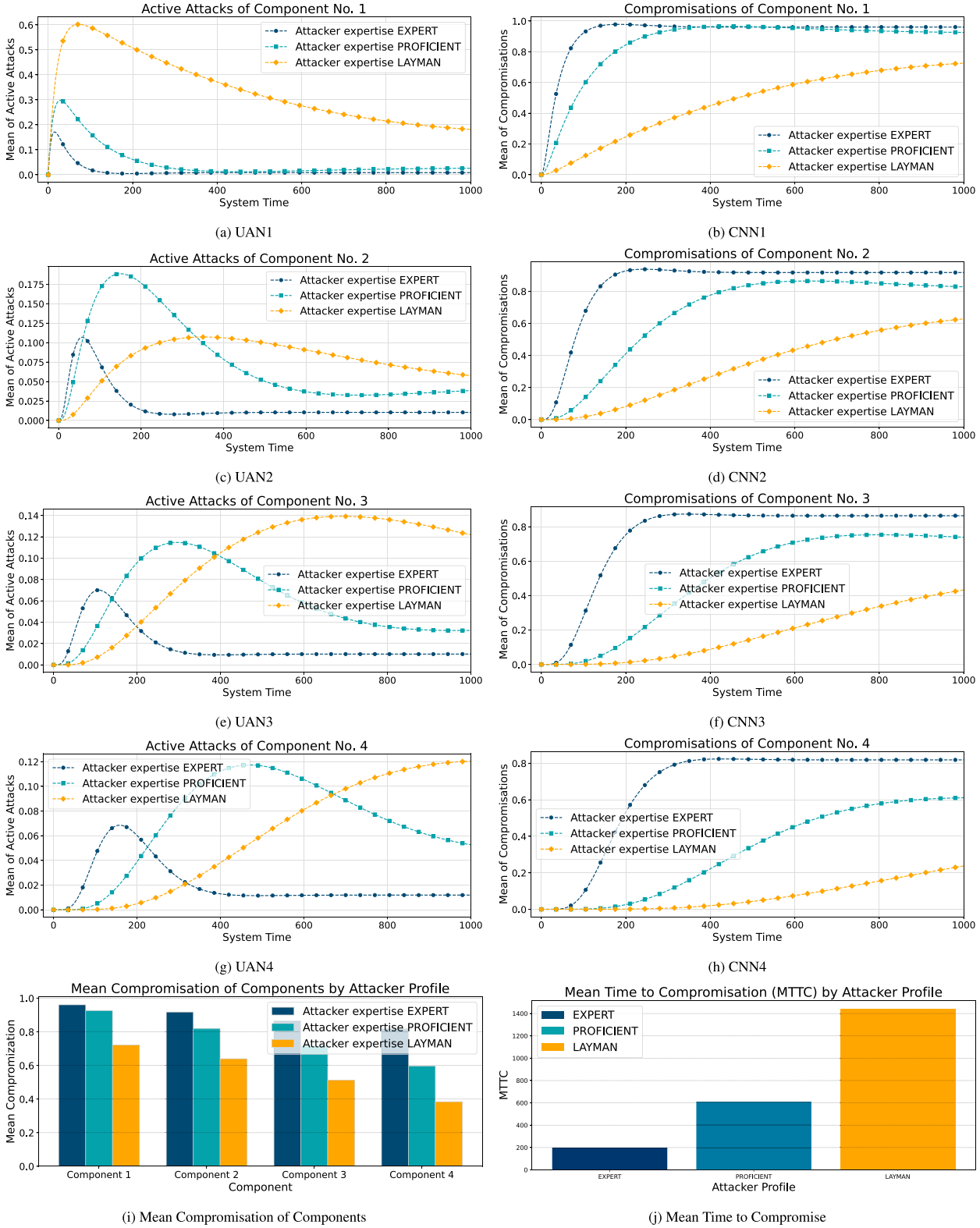

Fig. 10. Experimental results of transient analysis and steady-state solution for the BMW case study and Attack Path 1. Attacker profiles taken from EVITA — Expert, Proficient, and Layman.

the head unit with the highest CVSS base score to patch in order to observe how the overall risk changes. In practice, our proposed method can help choose the most suitable patch management approach. As for another defense measure, we implement an *IDPS* to enhance the system's security posture. Specifically, we assume that the IDPS is designed to detect and prevent malicious software updating behavior that may be initiated by an attacker who has physical access to the system. In our case study, this behavior is associated with the CVE-2018-9314 vulnerability. To model this, we change the vulnerability score of component *HU_C2* to 0.00. With recalculation of all the metrics,

the new risk scores are given in column $S3$ of Table 5. The overall risk score of this defense model is 9.10. Through this example, our proposed framework enables the identification of potential attack paths and the calculation of the overall security risk of the system. Additionally, by using the framework, one can assess and compare the effectiveness of different security strategies, helping to ensure the most effective security strategies are implemented. The analysis of our example demonstrates a significant reduction in the overall security risk. Using *Patching management*, the risk score is lowered from 13.63 → 5.83. For comparison, the *IDPS* approach merely lowers the risk score from 13.63 → 9.10. It follows from Eq. (3) that both approaches lowers the risk from *medium* to *low*.

To summarize, this case study provides insights into the early-stage security analysis of automotive in-vehicle networks using the proposed methodology. Applying the method to a real-world automotive example highlights perspectives from manufacturers, security analysts, and end-users. For automotive manufacturers, the method provides a structured way to model system architecture, identify risks and vulnerabilities early during the design phase, and support decision-making in selecting appropriate defense strategies when vulnerabilities are found. For security analysts, it offers a formal, quantitative framework for threat modeling, risk evaluation, and dynamic behavior analysis. While end-users do not directly interact with the method, they benefit from enhanced system trust through improved resilience. For security analysts, the method delivers a formal and quantitative framework for threat modeling, risk evaluation, and dynamic behavior analysis. Although end-users do not directly interact with the method, the result indirectly enhances trust through improved resilience. Overall, this application shows how the method effectively connects early-stage system modeling with dynamic security assessment.

5. Limitations and future work

While this study enables a broader examination of system resilience and facilitates early-stage cybersecurity assessment, some practical considerations remain. Effective application of the proposed method requires familiarity with UML-based system modeling, basic cybersecurity concepts, and formal modeling expertise for dynamic analysis using tools such as TimeNET. Apart from these general considerations, several specific limitations remain, as discussed below:

Abstract levels in system design process: Our model introduces a hierarchical structure; however, the optimal number of levels needed for effective security analysis remains uncertain. In our case study, we applied three levels of abstraction, which may not fully capture the complexity encountered in industrial practices. In practice, manufacturers or researchers with comprehensive knowledge of the system could create more detailed hierarchical models, such as incorporating an additional "functional layer" below the subsystem level. This would uncover additional risks, attack paths, and vectors while enabling deeper system analysis. However, creating such detailed models introduces challenges, including increased complexity, resource consumption, and time constraints that must be carefully considered. Future work will explore the application of our model across various network types and case studies to identify the most suitable levels of abstraction for robust security analysis. Future work will explore the application of our model across various network types and case studies to identify the most suitable levels of abstraction for robust security analysis.

Validation and Verification: This study validates our proposed method using a case study of an in-vehicle network. Nevertheless, further validation on a testbed that mirrors early-stage system development environments is essential. Additionally, a comprehensive comparison with other security models is needed to benchmark our approach's effectiveness within a broader context. Additionally, a comprehensive comparison of this approach with other security models is still needed to benchmark its effectiveness.

Incorporating with deterministic components: Our current model primarily uses stochastic elements, such as delay and probability factors, to represent uncertain behaviors. Future work could enhance the current DSPN framework by adding deterministic model components, like periodic security checks that trigger at set intervals. These checks would simulate routine maintenance, allowing us to assess how regular inspections affect system resilience and threat response. By including deterministic elements such as scheduled assessments or updates, the model would better reflect the role of proactive defense in preventing or mitigating attacks over time. This improvement would expand the model's extensibility, making it suitable for a wider range of security strategies, especially in systems that need constant monitoring.

6. Conclusions

This paper introduced an enhanced security assessment framework that integrates UML-based hierarchical attack modeling with dynamic security analysis through the use of DSPNs. This approach addressed a limitation in earlier models by incorporating dynamic analysis capabilities that can simulate evolving system states under various attack scenarios.

An automated procedure converts UML models to graph-based representations, which allows the calculation of key security metrics and an overall system security assessment. This process empowers system designers to identify potential security weaknesses early and proactively evaluate mitigation strategies.

By mapping UML models to DSPN structures, our approach bridges the gap between system architecture and dynamic security analysis. Our extended model incorporates stochastic events, such as time-based behaviors, delays, and probabilistic transitions. This capability is essential for capturing the timing and likelihood of attacks and system responses. It enables a realistic representation of both attack processes and defense mechanisms within the system.

The paper demonstrated the model's real-world relevance through a case study on an in-vehicle network, highlighting its applicability in complex environments like automotive systems.

CRediT authorship contribution statement

Tino Jungebloud: Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Nhung H. Nguyen:** Writing – review & editing, Writing – original draft, Methodology, Investigation. **Dan Dongseong Kim:** Writing – review & editing, Methodology, Conceptualization. **Armin Zimmermann:** Writing – review & editing, Methodology, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Tino Jungebloud reports financial support was provided by Federal Ministry for Economic Affairs and Climate Action. Tino Jungebloud reports financial support was provided by Carl Zeiss Foundation. Nhung H. Nguyen reports financial support was provided by Vingroup Science and Technology Scholarship. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is partially funded by the Federal Ministry for Economic Affairs and Climate Action (BMWK) as part of the projects RTAPHM (Real-Time Analytic, Prognostics and Health Management) and MISU (Model-based Development of Secure Digital Infrastructures for Service-Driven UAV Systems), reference numbers 20X1720A and 20X1736E. This work has received funding from the Carl Zeiss Foundation as part of the project Engineering for Smart Manufacturing (E4SM) under grant agreement no. P2017-01-005. This work is partially sponsored by the Vingroup Scholarship - Vingroup Science and Technology Scholarship Program for Overseas Study for Master's and Doctoral Degrees.

Data availability

Data will be made available on request.

References

Ajmone Marsan, M., Chiola, G., 1987. On Petri nets with deterministic and exponentially distributed firing times. In: Rozenberg, G. (Ed.), Advances in Petri Nets 1987. In: Lecture Notes in Computer Science (LNCS), vol. 266, Springer, pp. 132–145.

Bedini, F., Jungebloud, T., Maschotta, R., Zimmermann, A., 2024. An analysis and simulation framework for systems with classification components. In: Proceedings of the 12th International Conference on Model-Based Software and Systems Engineering. MODELSWARD, SciTePress, INSTICC, pp. 50–61. http://dx.doi.org/10.5220/0012357000003645.

Bonet, P., Llado, C.M., Puigjaner, R., 2007. PIPE v2.5: a Petri Net Tool for Performance Modeling. In: Proc. 23rd Latin American Conference on Informatics. CLEI 2007.

Cai, Z., Wang, A., Zhang, W., 2019. 0-days & mitigations: Roadways to exploit and secure connected BMW cars. pp. 1–37, BlackHat USA.

CVSS Special Interest Group (SIG), 2025. Common Vulnerability Scoring System v3.0: Specification Document. Forum of Incident Response and Security Teams, Inc (FIRST), North Carolina, USA, URL https://www.first.org/cvss/v3.0/specification-document.

Dalton, G.C., Mills, R.F., Colombi, J.M., Raines, R.A., 2006. Analyzing Attack Trees using Generalized Stochastic Petri Nets. 2006 IEEE Inf. Assur. Work. 116–123. http://dx.doi.org/10.1109/iaw.2006.1652085.

Enoch, S.Y., Ge, M., Hong, J.B., Kim, D.S., 2021a. Model-based Cybersecurity Analysis: Past Work and Future Directions. In: 2021 Annual Reliability and Maintainability Symposium. RAMS, pp. 1–7. http://dx.doi.org/10.1109/rams48097.2021.9605784, arXiv:2105.08459.

Enoch, S.Y., Hong, J.B., Ge, M., Kim, D.S., 2020. Composite Metrics for Network Security Analysis. http://dx.doi.org/10.48550/arxiv.2007.03486, arXiv arXiv:2007.03486.

Enoch, S.Y., Lee, J.S., Kim, D.S., 2021b. Novel security models, metrics and security assessment for maritime vessel networks. Comput. Netw. Volume 189, http://dx.doi.org/10.1016/j.comnet.2021.107934.

EUROCAE, 2014. ED-202A - Airworthiness Security Process Specification. Technical Report, The European Organisation for Civil Aviation Equipment, pp. 1–75, URL https://standards.globalspec.com/std/9862360.

EUROCAE, 2018. ED-203A - Airworthiness Security Methods and Considerations. Technical Report, The European Organisation for Civil Aviation Equipment, URL https://standards.globalspec.com/std/10393097.

Ge, M., Cho, J.-H., Kim, D., Dixit, G., Chen, I.-R., 2021. Proactive defense for Internet-of-things: Moving target defense with cyberdeception. ACM Trans. Internet Technol. 22 (1), 1–31. http://dx.doi.org/10.1145/3467021.

Ge, M., Hong, J.B., Guttmann, W., Kim, D.S., 2017. A framework for automating security analysis of the internet of things. J. Netw. Comput. Appl. 83, 12–27. http://dx.doi.org/10.1016/j.jnca.2017.01.033.

Groner, R., Witte, T., Raschke, A., Hirn, S., Pekaric, I., Frick, M., Tichy, M., Felderer, M., 2023. Model-based generation of attack-fault trees. In: Guiochet, J., Tonetta, S., Bitsch, F. (Eds.), Computer Safety, Reliability, and Security. Springer Nature Switzerland, Cham, pp. 107–120.

Hammer, M., Maschotta, R., Wichmann, A., Jungebloud, T., Bedini, F., Zimmermann, A., 2022. A model-driven implementation of PSCS specification for C++. Proceedings of the 9th International Conference on Model-Driven Engineering and Software Development, pp. 100–109. http://dx.doi.org/10.5220/0010267801000109.

Hong, J.B., Kim, D.S., 2012. HARMs: Hierarchical attack representation models for network security analysis. In: Australian Information Security Management Conference. pp. 74–81, URL https://api.semanticscholar.org/CorpusID:4678604.

Hong, J.B., Kim, D.S., 2016. Assessing the effectiveness of moving target defenses using security models. IEEE Trans. Dependable Secur. Comput. Volume 13 (Issue 2), 163–177. http://dx.doi.org/10.1109/tdsc.2015.2443790.

Jungebloud, T., 2025. C++ Header-only implementation of the Common Vulnerability Scoring System (CVSS) Version 3.1. URL https://github.com/dataliz9r/io.mbse.mora.score.cvss3.

Jungebloud, T., H. Nguyen, N., Seong Kim, D., Zimmermann, A., 2024. Hierarchical model-based cybersecurity risk assessment during system design. In: Meyer, N., Grocholewska-Czuryło, A. (Eds.), ICT Systems Security and Privacy Protection. Springer Nature Switzerland, Cham, pp. 30–44.

Lathrop, S.D., Hill, J.M.D., Surdu, J.R., 2003. Modeling network attacks. In: Proc. 12th Conf. Behavior Representation in Modeling and Simulation.

Leversage, D.J., Byres, E.J., 2008. Estimating a system's mean time-to-compromise. IEEE Secur. Priv. 6 (1), 52–60. http://dx.doi.org/10.1109/msp.2008.9.

Little, J.D.C., 1961. A proof of the queuing formula $L = \lambda W$. Oper. Res. 9, 383–387.

Maschotta, R., Silatsa, N., Jungebloud, T., Hammer, M., Zimmermann, A., 2022. An OCL implementation for model-driven engineering of C++. In: Software Engineering Research, Management and Applications. Springer International Publishing, Cham, pp. 151–168. http://dx.doi.org/10.1007/978-3-031-09145-2_10.

McQueen, M.A., Boyer, W.F., Flynn, M.A., Beitel, G.A., 2006. Quantitative cyber risk reduction estimation methodology for a small SCADA control system. In: Proceedings of the 39th Annual Hawaii International Conference on System Sciences, Vol. 9. HICSS'06, pp. 1–11. http://dx.doi.org/10.1109/hicss.2006.405.

Mendonça, J., Kim, M., Graczyk, R., Völp, M., Kim, D.D., 2022. Security modeling and analysis of moving target defense in software defined networks. In: 2022 IEEE 27th Pacific Rim International Symposium on Dependable Computing. PRDC, pp. 141–151. http://dx.doi.org/10.1109/prdc55274.2022.00028.

MITRE, 2023a. MITRE CAPEC - Common attack pattern enumeration and classification. URL https://capec.mitre.org.

MITRE, 2023b. MITRE CWE - Common weakness enumeration. URL https://cwe.mitre.org.

Monteuuis, J.-P., Boudguiga, A., Zhang, J., Labiod, H., Servel, A., Urien, P., 2018. SARA: Security Automotive Risk Analysis Method. In: Proceedings of the 4th ACM Workshop on Cyber-Physical System Security. pp. 3–14. http://dx.doi.org/10.1145/3198458.3198465.

Nie, S., Liu, L., Du, Y., 2017. Free-fall: Hacking tesla from wireless to CAN bUS. BlackHat USA.

OBEO, 2023. UML designer - graphical tooling to edit and visualize UML models. URL https://www.umldesigner.org/.

Olaf, H., Apvrille, L., Fuchs, A., Roudier, Y., Ruddle, A., Weyl, B., 2009. Security requirements for automotive on-board networks. In: 2009 9th International Conference on Intelligent Transport Systems Telecommunications. ITST, pp. 641–646. http://dx.doi.org/10.1109/itst.2009.5399279.

OMG, 2017. OMG Unified Modeling Language (OMG UML), Version 2.5.1. Object Management Group, URL https://www.omg.org/spec/UML/2.5.1/PDF.

OMG, 2019. OMG systems modeling language (OMG sysml), version 1.6. URL https://www.omg.org/spec/SysML/1.6/PDF.

Pedroza, G., 2019. Towards safety and security co-engineering: Challenging aspects for a consistent intertwining. In: Security and Safety Interplay of Intelligent Software Systems. Springer International Publishing, pp. 3–16. http://dx.doi.org/10.1007/978-3-030-16874-2_1.

Pedroza, G., Mockly, G., 2020. Method and framework for security risks analysis guided by safety criteria. In: Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings. pp. 1–8. http://dx.doi.org/10.1145/3417990.3420047.

Pekaric, I., Frick, M., Adigun, J.G., Groner, R., Witte, T., Raschke, A., Felderer, M., Tichy, M., 2023. Streamlining attack tree generation: A fragment-based approach. arXiv:2310.00654.

Roudier, Y., Apvrille, L., 2015. SysML-Sec - A model driven approach for designing safe and secure systems. In: 2015 3rd International Conference on Model-Driven Engineering and Software Development. pp. 655–664. http://dx.doi.org/10.5220/0005402006550664.

Shaked, A., Reich, Y., 2021. Model-based Threat and Risk Assessment for Systems Design. In: Proceedings of the 7th International Conference on Information Systems Security and Privacy. pp. 331–338. http://dx.doi.org/10.5220/0010187203310338.

SSE, 2023a. Model-driven software engineering for C++ (MDE4CPP). URL https://tinyurl.com/5n7rra64.

SSE, 2023b. UML designer - Technische Universität Ilmenau branch. URL https://github.com/MDE4CPP/UML-Designer.

Zimmermann, A., 2008. Stochastic Discrete Event Systems. Springer Berlin Heidelberg, http://dx.doi.org/10.1007/978-3-540-74173-2.

Zimmermann, A., 2017. Modelling and performance evaluation with TimeNET 4.4. In: Quantitative Evaluation of Systems - 14th Int. Conf.. QEST 2017, In: Lecture Notes in Computer Science, vol. 10503, Springer, Berlin, Germany, pp. 300–303.

Zimmermann, A., Jungebloud, T., 2023. Verbundvorhaben RTAPHM - Entwicklung leistungsbasierter Diensterbringung durch Digitalisierung und Optimierung der Plattformverfügbarkeit durch Datenanalytik und Prognose. Schlussbericht. Technische Universität Ilmenau, Fakultät für Informatik und Automatisierung, Fachgebiet System- und Software-Engineering, Ilmenau, http://dx.doi.org/10.2314/KXP:1896979300.